

[UNIVERSITY / COLLEGE NAME]

[FACULTY OF COMPUTER SCIENCE] — [DEEP LEARNING MAJOR]

CARA

Computer-Aided Review & Analysis of Facial Skin

A Multi-Model Deep Learning System for Automated Skin Assessment
and Evidence-Based Skincare Recommendation

Final Degree Project

B.Sc. in Computer Science — Deep Learning Track

Submitted by: [Student Name 1]
[Student Name 2]
[Student Name 3]
Supervisor: [Prof. / Dr. Supervisor Name]
Department: [Department Name]
Submission: [Month, Year]

CARA is not a medical diagnostic tool. It provides cosmetic skincare insights and educational ingredient recommendations only.

Abstract

Skincare decisions are today driven mostly by intuition, marketing, and influencers, while professional dermatological advice is expensive and often inaccessible. CARA is a web application that closes part of this gap by returning an honest, evidence-based assessment of a user’s facial skin from a single photograph, together with concrete ingredient-level recommendations.

The final system decomposes the assessment into three complementary classification tasks — acne severity, skin type, and multi-label skin conditions (pores, blackheads) — each handled by a dedicated EfficientNet-B0 convolutional neural network. The three predictions feed a deterministic, rule-based Business Logic Processing (BLP) engine that turns raw model outputs into actionable, explainable advice.

This report tells the honest story of how the system reached that design. The project went through three distinct phases. In Phase 1 we scoped the problem, surveyed competitors, and produced a proof of concept. In Phase 2 we compared four candidate architectures on our anchor datasets and chose three independent EfficientNet-B0 classifiers coupled to a rule-based fusion layer. In Phase 3 — the largest phase — real-world test images exposed catastrophic failures that a naive validation split had hidden: models were learning dataset signatures and photographic style rather than clinical features; a preprocessing pipeline that existed in code was silently disconnected in production; one of the “healthy” training folders turned out to be *acne-level-zero* photographs with visible blemishes; and a softmax over five skin issues was mathematically incapable of saying “none”. We responded by rebuilding the dataset from the ground up, reframing skin issues as a multi-label problem with proper negative anchors, enforcing training–inference preprocessing parity, and adapting the model to in-the-wild photographs by borrowing negatives from an orthogonal dataset.

The final acne and skin-type classifiers reach 72.9 % and 70.9 % best validation accuracy on their respective test splits; the multi-label skin-conditions classifier reaches a macro-F1 of **0.981** after the domain-adaptation retrain. Beyond raw accuracy, our contribution is the emphasis on *truthfulness*: the guarantee that when the system is uncertain, it says so, and that every recommendation shown to the user can be traced to a single, inspectable rule.

Keywords: deep learning, computer vision, convolutional neural networks, EfficientNet, transfer learning, image classification, multi-label classification, dermatology, skincare, rule-based systems, explainable AI, domain adaptation, FastAPI, React.

Acknowledgements

We thank our supervisor, [Supervisor Name], for the guidance, the patience, and the pointed feedback that pushed the project from a demo into a system we are willing to stand behind. We thank the members of the [Department Name] for the technical infrastructure and the open-door policy whenever we hit a wall. Finally, we thank our families and friends for tolerating months of conversations that revolved around loss curves, class weights, and skin types.

Contents

Abstract	i
Acknowledgements	ii
List of Figures	viii
List of Tables	ix
1 Introduction and Motivation	1
1.1 The problem we set out to solve	1
1.2 Why this problem is a good fit for deep learning	1
1.3 Non-negotiable design principles	2
1.4 The three phases at a glance	2
1.5 Contributions	3
1.6 Reading guide	3
2 Background and Related Work	4
2.1 The clinical baseline	4
2.2 Public datasets we evaluated	4
2.3 Related academic work	5
2.3.1 Lesion classification on dermoscopic images	5
2.3.2 Acne severity grading	5
2.3.3 Skin-type and cosmetic assessment	5
2.3.4 Multi-label medical image classification	5
2.4 Commercial and industrial products	6
2.5 Where CARA fits in the picture	6
3 Academic Background	7
3.1 Image classification, formally	7
3.2 Multi-label classification	7
3.3 Convolutional neural networks	8
3.4 From AlexNet to EfficientNet	8
3.5 Transfer learning	9
3.6 Regularisation and augmentation	9
3.7 Class imbalance	9
3.8 Optimisation	10

3.9	Rule-based systems and BLP	10
3.10	Softmax, cross-entropy and their gradient	10
3.11	Batch normalisation	11
3.12	Metrics beyond accuracy	11
4	The Data	12
4.1	Phase 1: the initial data landscape	12
4.2	Phase 2: consolidation on cleaner anchor data	12
4.3	Phase 3: the audit and the rebuild	13
4.3.1	The acne dataset: bounding boxes were not the point	13
4.3.2	The “healthy” folder that was not healthy	13
4.3.3	The “wrinkles” folder that was elderly stock photos	14
4.3.4	Softmax cannot say “none”	14
4.3.5	The preprocessing pipeline that was dead code	14
4.4	Phase 3: the final dataset layout	14
4.4.1	Acne severity split and weights	14
4.4.2	Skin-type split and weights	15
4.4.3	Skin-conditions dataset (multi-label)	15
4.4.4	BCE positive weights for skin conditions	16
4.5	Splits and reproducibility	17
4.6	Cleaning	17
4.7	Augmentation	17
4.8	The label agreement problem	18
5	Our Algorithms	19
5.1	The final pipeline, end to end	19
5.2	Why EfficientNet-B0, and why three of them	20
5.3	Two-phase transfer learning	20
5.4	Phase 2: comparing four candidate architectures	21
5.4.1	Candidate 1 — single multi-task EfficientNet-B0	21
5.4.2	Candidate 2 — three independent EfficientNet-B0 + weighted-vote fusion	22
5.4.3	Candidate 3 — three independent EfficientNet-B0 + learned MLP fusion	22
5.4.4	Candidate 4 — three independent EfficientNet-B0 + rule-based BLP .	23
5.4.5	Summary of the Phase-2 comparison	23
5.5	Phase 3, part 1: real-world failures	23
5.6	Preprocessing parity	24
5.6.1	Face detection and cropping	24
5.6.2	The dead-code discovery	25
5.6.3	EXIF orientation and rotation retry	25
5.7	Phase 3, part 2: refactoring skin issues to multi-label	26
5.7.1	Downstream code changes	26

5.8	Phase 3, part 3: the domain-adaptation retrain	27
5.8.1	The creative fix: cross-task negative mining	27
5.8.2	The retrain and the honest numbers	28
5.8.3	Why this is defensible	29
5.9	Confidence and uncertainty	29
5.10	The BLP recommendation engine	29
5.10.1	Rule design principles	30
5.10.2	Coverage over the label spaces	30
5.11	Codebase sanitation	31
5.12	Backpropagation, in one page	31
6	Experiments	33
6.1	Experimental setup	33
6.2	Failed experiment: naive resizing without face crop	33
6.3	Failed experiment: unweighted cross-entropy	34
6.4	Failed experiment: aggressive augmentation	34
6.5	Successful comparison: two-phase versus single-phase training	34
6.6	Successful comparison: backbone consideration	35
6.7	The real-world regression suite	35
7	Results and Evaluation	37
7.1	Final headline numbers	37
7.2	Interpreting the numbers	37
7.3	Real-world regression results	38
7.4	Qualitative case studies	38
7.5	Latency and resource profile	39
8	The Product	40
8.1	User story	40
8.2	The landing page and disclaimer	40
8.3	Authentication	40
8.4	Analyze — upload and live capture	42
8.4.1	File-picker flow	42
8.4.2	Live-camera capture with face lock	42
8.5	Results	42
8.6	History and progress tracking	43
8.7	System architecture	43
8.8	Frontend	49
8.9	Backend	49
8.10	Storage	49
8.11	Deployment	50

9	Software Engineering, Testing, and MLOps	51
9.1	Repository layout	51
9.2	Testing pyramid	51
9.2.1	Unit tests	52
9.2.2	Integration tests	52
9.2.3	Model accuracy tests	52
9.2.4	End-to-end tests	52
9.2.5	Real-world regression tests	53
9.3	Reproducibility	53
9.4	Debugging techniques we relied on	53
9.5	Configuration and secrets	53
9.6	Checkpoint management	53
10	Discussion	55
10.1	Interpreting the story	55
10.2	What worked	55
10.3	What did not work	55
10.4	Threats to validity	56
10.5	Ethical considerations	56
11	Conclusions	57
12	Future Work	58
12.1	Better data for the minority classes	58
12.2	Ordinal regression for acne severity	58
12.3	Attention-based visualisation	58
12.4	On-device inference	58
12.5	Larger regression set with diverse skin tones	58
12.6	Formal dermatologist review of the rules	59
A	API reference	62
B	Report schema	63
C	Reproducibility recipe	64
D	Ingredient reference	65
E	Glossary	67

List of Figures

3.1	Schematic of the EfficientNet-B0 backbone used for each of CARA’s three classifiers. The classifier head (dropout + fully-connected + softmax or sigmoid) is the only part re-initialised for each task.	9
5.1	End-to-end runtime pipeline used by CARA at submission time. The three EfficientNet-B0 models run sequentially on a shared preprocessed tensor; the BLP engine composes their outputs deterministically. Every arrow in the diagram was, at some point, a bug — see Sections 5.6, 5.7 and 5.10.	19
5.2	Actual per-epoch validation accuracy on the acne severity task, from <code>training_log.txt</code> . The dashed line marks the best-saved checkpoint.	21
5.3	Per-epoch validation macro- F_1 for both skin-conditions retrains, from <code>training_log.txt</code> . Retrain #1 peaks slightly higher on its own test split but generalises worse to real-world photographs; see Section 5.8.	22
7.1	Best validation metric per task. Bars for acne severity and skin type report accuracy; the skin-conditions bar reports macro- $F_1 \times 100$. The metrics are not on the same footing — see Section 7.2.	37
8.1	The CARA landing page. The “Educational Use Only” banner at the bottom is a first-class part of the layout, not a footer.	41
8.2	The login page. Attempting to reach a protected page without a session redirects here.	41
8.3	The Analyze page. Tips for a good photograph are surfaced directly in the flow.	42
8.4	File-picker flow with a selected image ready to analyse.	43
8.5	Live camera capture with lighting / face / position / angle checks. The button unlocks only when all four are green.	44
8.6	Results page, top half. Each model has its own card with a per-model confidence bar. Skin Conditions is a multi-label card that lists findings (here: one finding, “Blackheads”).	45
8.7	Results page, bottom half. The ingredient list is grouped by ingredient category. The Educational Note collects the plain-English notes from each rule that fired.	46
8.8	History page. The list of reports below the chart is clickable — each row leads back to the full recommendation page for that historical report.	47
8.9	Chart point tooltip on the History page.	47
8.10	End-to-end runtime architecture of CARA.	48

List of Tables

2.1	Positioning of CARA relative to representative existing products.	6
4.1	Final training datasets used by CARA. Splits are 70 / 15 / 15 (train / val / test) for the datasets we split ourselves; <code>skin_type</code> came pre-split by the original authors.	15
4.2	Class distribution and derived per-class weights for the acne severity training set. Weights follow the balanced scheme in Equation (3.4), and match the numbers used by our training script.	15
4.3	Split sizes for the acne severity task.	15
4.4	Split sizes for skin type.	16
4.5	Class distribution and derived per-class weights for skin type.	16
4.6	Classes dropped from the Phase-2 skin-issues dataset, with reasons.	16
4.7	Composition of <code>data/skin_conditions/</code> (Phase-3 retrain #2). “skintype_” files are prefixed on disk so each is traceable to its source class.	17
5.1	Candidate architectures compared during Phase 2. The fourth row is what we ship.	23
5.2	The multi-label refactor at a glance.	26
5.3	Real-world regression on the multi-label skin-conditions model, before and after the domain-adaptation retrain.	28
5.4	Rule coverage over the three label spaces, from <code>backend/blp/rules.json</code>	31
6.1	Two-phase versus single-phase training on the acne severity task. Numbers are best validation accuracy.	34
6.2	Backbone consideration. Params and FLOPs are from the published papers; latency is approximate on our reference CPU. We did not build the full ResNet-50 pipeline because its inference latency did not fit our budget.	35
7.1	Final validation metrics for the three models, from <code>training_log.txt</code>	37
7.2	Real-world regression outcomes before and after the Phase-3 rework. “Population sanity” rows are boolean: either the model can produce the class at least once on the folder or it cannot.	38
7.3	Representative end-to-end reports.	39
7.4	Approximate wall-clock latency breakdown of a single <code>/api/analyze</code> request. Reference hardware: mid-range laptop CPU, warm cache. The three model inferences run sequentially in the current implementation.	39

9.1	Composition of the CARA test suite.	52
D.1	Frequently used ingredients in the CARA recommendation vocabulary.	66

Introduction and Motivation

1.1 The problem we set out to solve

Skin is the largest organ in the human body, and yet the decisions people make about their skin are among the most poorly informed decisions in everyday life. Consumers spend billions each year on skincare products chosen almost entirely on the basis of marketing, celebrity endorsements, and viral trends. Dermatologists are expensive and overbooked; general practitioners rarely have time for cosmetic concerns; and the loudest voices online are often those with the least qualifications.

The result is a market in which a person with mild acne might spend a year using a moisturiser designed for dry, ageing skin, while another person with sensitive combination skin scrubs their face with an abrasive salicylic-acid cleanser marketed for severe cystic acne. Neither person is lazy or foolish. They simply lack access to a truthful, personal, and actionable assessment of their own skin.

CARA— *Computer-Aided Review & Analysis* — is our attempt to provide a small slice of that missing information. The goal is deliberately narrow: given a single facial photograph uploaded through a web browser, return within a few seconds an honest report describing the user’s acne severity, skin type, and any notable skin conditions, together with a concrete list of ingredients to look for and ingredients to avoid.

1.2 Why this problem is a good fit for deep learning

Dermatological image classification has three properties that make it particularly interesting from a deep-learning point of view:

1. **Highly visual signal.** Almost everything that a dermatologist evaluates in a routine cosmetic consultation is visible in a well-lit photograph: comedones, papules, pustules, pore density, oily sheen, dry flakes, hyperpigmentation.
2. **Weak, noisy labels.** Public datasets are labelled by different clinicians, with different rating scales, in different lighting conditions. This forces us to think seriously about preprocessing, augmentation, and class imbalance rather than assuming clean data.
3. **Multiple correlated but distinct sub-tasks.** Acne severity, skin type, and skin

conditions are related — oily skin is more prone to acne, blackheads occur where pores are wide — but they are not the same task. This creates a natural design question: one big model or several small ones?

1.3 Non-negotiable design principles

Very early in the project we agreed on three rules that would govern every technical decision. We refer back to them throughout the report.

Truthfulness is the product. A wrong prediction is worse than no prediction. If the model is not confident, the system must say so out loud rather than paper over the uncertainty.

No manipulated results. We never tune a threshold to make an embarrassing prediction disappear. Numbers reported in this book are the numbers we measured.

Explainability comes from design, not post-hoc excuses. The rule engine that converts model predictions into recommendations is deterministic and human-readable. A user (or a supervisor) can trace exactly why a specific recommendation was made.

These principles are not decorative. They ruled out several attractive architectures — learned recommenders, ensemble score fusion, threshold calibration on the test set — and they are the reason the project took three phases rather than one to reach its current shape.

1.4 The three phases at a glance

The rest of this book is organised topically rather than chronologically, but the reader will encounter three distinct phases of work throughout:

Phase 1 — Proof of concept and scoping. Competitor analysis, dataset survey, a first pipeline that ran end-to-end on a laptop. This phase committed us to the “three-model + rules” shape of the system without yet knowing which architecture inside each model would win.

Phase 2 — Architecture selection on our anchor data. We compared four candidate architectures (a single multi-task network, three independent CNNs with weighted-vote fusion, three independent CNNs with a learned MLP fusion, and three independent CNNs with a rule-based engine) on our then-current datasets. The rule-based engine coupled to independent EfficientNet-B0 classifiers won. At the end of Phase 2 we had strong validation numbers and a growing sense that they meant less than they looked.

Phase 3 — Real-world reckoning and rework. Trials on actual user selfies revealed catastrophic failures that training-split validation had hidden. We audited every dataset, discovered several were fundamentally mislabelled or of the wrong task type, and re-built: we replaced the acne dataset, deprecated the old skin-issues model, moved skin

conditions to a multi-label formulation with proper negative anchors, wired a preprocessing pipeline that had been dead code, and used cross-task data reuse to close the domain gap between our training distribution and real user photos.

Phase 3 is where most of the depth of this book lives. It is also, not coincidentally, the phase in which our three non-negotiable rules did the most work.

1.5 Contributions

The concrete contributions of this project are:

- A production-grade, end-to-end web application that runs three deep-learning models in parallel and returns a report within a couple of seconds on CPU-only hardware.
- A comparative study of four candidate architectures with an honest discussion of why three of them failed, resulting in a design that trades a small amount of raw accuracy for a large amount of trust and explainability.
- A multi-label refactor of the skin-conditions problem, with cross-task negative-anchor mining, that raised out-of-domain precision without a fresh labelling effort.
- A three-layer regression test suite — per-image assertions, population-level sanity checks, and multi-label specific assertions — that catches the class of bug that ordinary train/val/test evaluation is structurally blind to.
- An honest, defensible narrative of every experiment we ran, including the ones that failed and why they failed.

1.6 Reading guide

The book is organised so that a reader can either follow it end-to-end or enter at whichever chapter interests them. Chapter 2 surveys the academic and industrial landscape. Chapter 3 covers the deep-learning background needed to follow the technical chapters. Chapter 4 describes the datasets, how we cleaned them, and how they evolved across the three phases. Chapter 5 is the main technical chapter, walking through the architecture, the four Phase-2 candidates, and the Phase-3 rework in detail. Chapters 6 and 7 report experiments and final results. Chapter 8 describes the product from a user’s point of view, with screenshots. Chapter 9 covers the software engineering and testing practice around the models. Chapters 10 to 12 close the book with discussion, conclusions, and directions for future work.

Background and Related Work

2.1 The clinical baseline

Modern dermatology grades acne along ordinal scales such as the Investigator’s Global Assessment (IGA) or the Global Acne Grading System (GAGS). Both scales collapse a rich visual scene into a small number of categories (typically *clear*, *mild*, *moderate*, *severe*). Skin type is described using variants of the classical four-way split *oily*, *dry*, *normal*, *combination*. Secondary conditions — blackheads, dark spots, enlarged pores, wrinkles — are usually discussed qualitatively rather than graded on a formal scale.

This clinical vocabulary matters because it defines the labels our models must produce. Inventing new categories would make it impossible for a domain expert to validate the output. We therefore aligned our label spaces with the categories a dermatologist would recognise.

2.2 Public datasets we evaluated

Progress in dermatological deep learning has been paced by dataset releases. The most influential public resources we considered:

- **ACNE04** [12]. Roughly 1,400 facial images labelled with acne severity plus per-lesion bounding boxes. The original paper formulates the task as label distribution learning; the class labels can also be used directly. Photographs are in-the-wild and vary widely in lighting and pose, which is both a curse and a blessing: they are noisy but they resemble real user uploads. *Our anchor dataset for acne severity.*
- **Kaggle Face Skin Type / Skin Problems datasets**. A family of smaller, community-contributed datasets covering skin type and skin problem categories. Quality varies but they fill gaps left by the more curated academic datasets.
- **ISIC / HAM10000** [13]. A large collection of dermoscopic images labelled with skin-lesion categories. Excellent for melanoma / lesion classification but not directly applicable to cosmetic assessment because the images are dermoscopic close-ups rather than casual photographs.

- **Fitzpatrick 17k** [14]. A dataset of clinical photographs annotated with Fitzpatrick skin type, used mostly for fairness studies. Referenced but not used for training.
- **Roboflow object-detection sets for acne and skin conditions**. Detection-annotated sets with bounding boxes around lesions or comedones. These looked useful in Phase 1 but caused us serious problems in Phase 3, as Chapter 4 details.

2.3 Related academic work

Deep learning for skin analysis divides roughly into three lines of work.

2.3.1 Lesion classification on dermoscopic images

The best-known result is Esteva et al. [11], who trained a single Inception-v3 network on a large private dataset of clinical images and reached dermatologist-level performance on binary melanoma classification. Subsequent work explored specialised architectures such as EfficientNet [7], ResNeXt [6], and Vision Transformers, plus data strategies such as heavy augmentation, colour-constancy normalisation, and ensemble averaging.

2.3.2 Acne severity grading

The ACNE04 paper [12] formulates acne severity as a label-distribution learning problem, arguing that hard categorical labels lose information (a photograph labelled *moderate* is often only marginally different from one labelled *mild*). Follow-up work has used ordinal regression, multi-task lesion counting, and attention-based localisation. For our project we deliberately chose the simpler categorical formulation, because the downstream consumer of the prediction — the rule engine — expects discrete severity buckets.

2.3.3 Skin-type and cosmetic assessment

Skin-type classification has received less academic attention than lesion classification, and most published pipelines come from industry rather than academia. We were unable to find a widely-cited academic benchmark for the four-class oily / dry / normal / combination task. This is one reason our own results on skin type are harder to compare to external numbers than our acne results are.

2.3.4 Multi-label medical image classification

Because acne, pores, and blackheads can co-occur on the same face, we also drew on the multi-label medical-imaging literature. The architecture pattern we ultimately adopted for skin conditions — independent sigmoid outputs trained with binary cross-entropy and per-class F_1 evaluation — is standard in chest-X-ray classification [16] and translates cleanly to our setting.

2.4 Commercial and industrial products

Several commercial products have appeared in the last few years that occupy roughly the same niche as CARA. Each one influenced our design either by example or by counter-example.

- **L’Oréal Skin Genius / ModiFace.** A polished mobile experience that returns a numerical score and product recommendations. The recommendations are, unsurprisingly, drawn exclusively from the vendor’s own catalogue. This motivated our decision to recommend *ingredients* rather than branded products.
- **Perfect Corp. / YouCam Makeup.** Very strong face tracking and augmented-reality effects. Their skin-quality assessment is fast but has been criticised for optimistic outputs that appear to be tuned for engagement rather than accuracy. This informed our stance on truthfulness.
- **Curology.** A subscription tele-dermatology service in which real clinicians write prescriptions after reviewing user photographs. Not an AI product per se, but the reference for what a truthful assessment looks like.
- **Academic demos.** A number of open-source projects on GitHub attempt facial skin analysis with pretrained CNNs. Almost all of them fall into one of two failure modes: they return a confident label on every input, including images with no face; or they degrade to noise as soon as lighting changes. Both failures shaped the guardrails in our pipeline.

Table 2.1: Positioning of CARA relative to representative existing products.

	L’Oréal SG	Perfect Corp.	Curology	Cara
Delivery	Mobile app	Mobile app	Human tele-derm	Web app
Assessment	Learned score	Learned score	Human	3 CNNs + rules
Recommends	Own products	Own products	Prescription	Ingredients
Explainability	None	None	Full	Rule engine
Handles unsure	Rarely	Rarely	Always	By design
Open source	No	No	No	Yes

2.5 Where Cara fits in the picture

CARA sits deliberately in the small corner of the market that combines (i) open-source, reproducible, deep-learning components, with (ii) a transparent, rule-based recommendation layer, and (iii) an honest policy about uncertainty. We do not compete with L’Oréal on polish, and we do not compete with Curology on medical rigour. We compete with the countless “*free online skin analysis*” demos on the accuracy and honesty of the report they return.

Academic Background

This chapter compresses the deep-learning material that the rest of the book relies on. Readers already familiar with convolutional neural networks, transfer learning, multi-label classification, and standard training practice can skip straight to Chapter 4.

3.1 Image classification, formally

A single-label image classifier is a function $f_\theta : \mathcal{X} \rightarrow \Delta^{K-1}$ parameterised by weights θ , where $\mathcal{X}$ is the set of input images and Δ^{K-1} is the probability simplex over K classes. Training minimises the empirical (weighted) cross-entropy loss

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K w_k y_{i,k} \log f_\theta(x_i)_k \quad (3.1)$$

on a dataset $\{(x_i, y_i)\}_{i=1}^N$, where y_i is a one-hot target and w_k is a per-class weight used to counteract class imbalance (Section 3.7).

3.2 Multi-label classification

Two of the three tasks in CARA are single-label (acne severity and skin type: the categories are mutually exclusive by definition). The third — skin conditions — is multi-label: a face can have blackheads AND enlarged pores simultaneously, or neither. A softmax over five categories, as we discovered in Phase 3, is mathematically incapable of representing “neither”.

A multi-label classifier replaces the softmax head with independent sigmoid outputs, one per class. Training minimises binary cross-entropy (BCE) with an optional positive weight:

$$\mathcal{L}_{\text{BCE}}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K \left[p_k y_{i,k} \log \sigma(z_{i,k}) + (1 - y_{i,k}) \log(1 - \sigma(z_{i,k})) \right] \quad (3.2)$$

where σ is the sigmoid, $z_{i,k}$ the logit for class k on sample i , $y_{i,k} \in \{0, 1\}$ the multi-hot target, and p_k the positive weight (used to counter class imbalance per output). Prediction is a per-class threshold, typically 0.5. Because the loss decomposes over classes, the model

can output any subset of labels, including the empty set — and this is exactly why the multi-label formulation was the right fit for skin conditions.

3.3 Convolutional neural networks

CNNs [1] exploit two properties of images: translation equivariance (a face is still a face when it moves five pixels to the right) and locality (nearby pixels are correlated). A convolutional layer computes

$$Y_{c,i,j} = \sigma \left(b_c + \sum_{c'} \sum_{u,v} K_{c,c',u,v} X_{c',i+u,j+v} \right) \quad (3.3)$$

for each output channel c , spatial position (i, j) and non-linearity σ . Stacking convolutional layers with pooling and non-linearities produces a hierarchy of features: edges and textures in the shallow layers, object parts in the middle layers, and semantic categories near the head.

3.4 From AlexNet to EfficientNet

The past decade of image classification is essentially a story of progressively better trade-offs between depth, width, resolution, and compute:

- **AlexNet** [2] (2012) proved that a deep CNN on GPU could crush hand-crafted features on ImageNet.
- **VGG** [3] pushed depth using small 3×3 filters.
- **ResNet** [5] introduced residual connections that allowed training networks with hundreds of layers.
- **Inception** [4] and its successors varied filter sizes in parallel.
- **EfficientNet** [7] performed a *compound scaling* of depth, width, and input resolution subject to a fixed compute budget, producing a family of models that dominated the accuracy–FLOP frontier.

We chose the smallest member of that family, EfficientNet-B0, for reasons that are worth stating explicitly.

Observation 3.1. *EfficientNet-B0 has approximately 5.3 million parameters. Our fine-tuned version has about 4.01 million parameters (with a small custom head of two classes, four classes, or five classes depending on the task). At 224×224 input it requires roughly 0.39 GFLOPs per forward pass. That budget is small enough to serve three parallel models on a laptop CPU in under two seconds, which is our performance target. This budget is*

not incidental to the choice — the entire project was designed to run on our own machines during training.

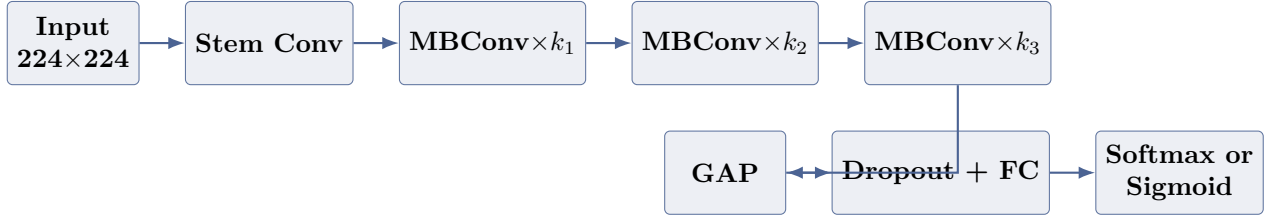


Figure 3.1: Schematic of the EfficientNet-B0 backbone used for each of CARA’s three classifiers. The classifier head (dropout + fully-connected + softmax or sigmoid) is the only part re-initialised for each task.

3.5 Transfer learning

Training a modern CNN from random weights on 1,000 images is a recipe for overfitting. Transfer learning [9] solves this by initialising the network with weights pretrained on a large source dataset — for us, ImageNet-1k [8] — and then adapting the model to the target task. Two knobs control the process:

- Which layers to freeze and for how long.
- How the learning rate differs between the pretrained backbone and the newly-initialised head.

Our two-phase recipe (Section 5.3) is a straightforward instance of this pattern.

3.6 Regularisation and augmentation

Small datasets punish confident networks. We rely on three regularisers:

1. **Data augmentation.** Random resized crop, horizontal flips, mild rotations, colour jitter. All augmentations are applied to training data only, never to validation or test data.
2. **Dropout** ($p = 0.3$) in the classifier head.
3. **Weight decay** via AdamW [10].

3.7 Class imbalance

Real-world skin datasets are heavily skewed. In our final acne dataset the *severe* class contains roughly one seventh as many samples as *mild*. Naive training would produce a classifier that predicts *mild* everywhere and reports respectable accuracy.

For single-label tasks we use weighted cross-entropy with weights derived from sklearn’s balanced weighting rule:

$$w_k = \frac{N}{K \cdot n_k} \quad (3.4)$$

where N is the total number of training samples, K the number of classes, and n_k the number of samples in class k . The weights we actually used are listed in Chapter 4.

For multi-label tasks (Section 3.2) we use the positive weight $p_k = n_k^{\text{neg}}/n_k^{\text{pos}}$ inside BCE, clipped to $[0.1, 10]$ to keep the loss numerically stable.

3.8 Optimisation

We use AdamW with an initial learning rate of 10^{-3} for the classifier head and 10^{-4} for the unfrozen backbone. Learning rate scheduling is cosine annealing with a short linear warm-up. Early stopping is triggered by validation loss (or, for the multi-label model, validation macro- F_1), not by training loss.

3.9 Rule-based systems and BLP

The last piece of academic background is on the recommendation side. Business Logic Processing (BLP) is our internal name for the data-driven, deterministic rule engine that converts model outputs into user-facing advice. It is closer in spirit to a decision table than to a formal rule system such as Datalog, but the important point is that every recommendation is produced by evaluating a hand-written rule against the model’s structured prediction. There is no learning component in the recommendation step, by design: any recommendation shown to the user can be justified by pointing at a single entry in a JSON file. Section 5.10 describes the engine we ultimately built.

3.10 Softmax, cross-entropy and their gradient

Every one of our single-label heads ends with a softmax layer

$$p_k = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)}, \quad (3.5)$$

whose input z is the logit vector produced by the final linear layer. Cross-entropy loss against a one-hot target y evaluates to $\mathcal{L} = -\log p_{k^*}$ where k^* is the index of the true class. A short calculation gives the intuitive gradient $\partial \mathcal{L} / \partial z_k = p_k - y_k$, which is the reason the softmax-cross-entropy combination is numerically and computationally so pleasant.

3.11 Batch normalisation

Every stage of an EfficientNet contains batch normalisation layers [15], which whiten the intermediate activations before the non-linearity:

$$\hat{a}_c = \gamma_c \cdot \frac{a_c - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}} + \beta_c. \quad (3.6)$$

Two consequences matter for our project. First, batch norm reduces the sensitivity to the initial learning rate, which stabilised fine-tuning in our two-phase recipe. Second, the running mean and variance are estimated on the fly during training and then frozen at inference time; forgetting to switch the model to `eval()` mode before inference was, in fact, one of our early Phase-2 bugs.

3.12 Metrics beyond accuracy

We report accuracy for single-label tasks and macro- F_1 for the multi-label task because those are the metrics a non-expert reader expects. Internally we track:

- **Macro-averaged F_1 .** The unweighted mean of the per-class F_1 scores. Insensitive to class imbalance.
- **Balanced accuracy.** The unweighted mean of per-class recall. Comparable to accuracy on balanced data but more honest on imbalanced data.
- **Confusion matrix.** The full per-class breakdown. Any conversation about where a model is failing starts here.
- **Real-world regression pass rate.** The percentage of our hand-labelled in-the-wild photographs on which the model predicts correctly. This turned out to be a much better proxy for shipped quality than any validation-split number.

The final bullet is a lesson from Phase 3: no matter how carefully constructed, a validation split of a training set only proves that the model can memorise its own distribution. Section 6.7 describes the regression suite we built to measure the real thing.

The Data

Data is the single largest determinant of model quality in this project, and by a wide margin. This chapter documents where the data came from, what we discovered when we looked closely, and how the datasets evolved across the three phases. The evolution is important because the *same* model trained on the *same* architecture gave completely different real-world behaviour before and after each data intervention.

4.1 Phase 1: the initial data landscape

We began with an ambitious plan: five models, one for each of five skin aspects (acne severity, skin type, blackheads, pores, dark spots). For each aspect we searched public repositories for the first dataset that seemed to match. Two Phase-1 choices later turned out to be catastrophic:

1. The initial acne dataset was an **object-detection** dataset — photographs annotated with bounding boxes around individual pimples. The idea was to run an object detector, count boxes, and derive severity from the count. In practice this meant almost no “clear” examples (nobody annotates healthy faces with zero boxes), and the signal the eventual classifier learned was “how much stuff got detected”, not “what is the clinical severity”.
2. The initial “healthy” folder inside the skin-issues dataset consisted of images with the filename prefix `level0_*.jpg` — level-0 acne from a different dataset. Level 0 is *not* the same as clean skin. Those photographs actually contain visible pimples and blackheads, just less severely. Trained as a “healthy” class it taught the model exactly the wrong concept.

Neither problem was visible in Phase 1 because Phase 1 evaluations were the classical train / validation / test splits of the same folders. The model happily memorised the labels it was given.

4.2 Phase 2: consolidation on cleaner anchor data

After the proof of concept we tightened the sourcing rules. Every dataset had to be a classification dataset (never a detection set repurposed as classification), had to have visible

labels for every class we cared about, and had to be spot-checked by a human before being accepted. We settled on the following combination for our four tasks (five, temporarily, before Phase 3 collapsed skin issues down to a single multi-label model):

- **Acne severity** — ACNE04 [12], used purely as a classification dataset (labels only, bounding boxes discarded).
- **Skin type** — a Kaggle community dataset with four classes (oily / dry / normal / combination), split into train / valid / test folders.
- **Skin issues (Phase 2 version)** — a merged Roboflow dataset with five classes (blackheads / dark spots / healthy / pores / wrinkles). *This is the dataset that blew up in Phase 3.*

Phase 2 numbers looked strong on their own validation splits. It was only when we started running real user selfies through the pipeline that we realised those numbers were measuring the wrong thing.

4.3 Phase 3: the audit and the rebuild

Phase 3 began with a single failing photograph: a perfectly clean, well-lit adult selfie that came back as *severe acne, oily skin, wrinkles detected*. Every part of that answer was wrong. Debugging that single image took us through a full audit of the data, the preprocessing pipeline, and the loss functions. The rest of this section is what we found.

4.3.1 The acne dataset: bounding boxes were not the point

Section 5.3 above already flagged the Phase 1 object-detection problem. In Phase 2 we had already replaced the detection-annotated data with ACNE04 as a classification dataset, so this specific problem was fixed by then — but only for acne. The lesson stuck: if the dataset’s original intent is a different task, do not simply re-use it.

4.3.2 The “healthy” folder that was not healthy

The Phase-2 skin-issues dataset contained a `healthy/` subfolder whose images were named `levle0_*.jpg`. Those photographs came from an external acne-severity level-0 collection. Level 0 is the lowest acne severity, but not the absence of blemishes — inspection showed visible pimples, blackheads, and mild dark spots in most of the “healthy” files.

Trained on this folder as a positive example of health, the model learned the wrong concept. Empirically it *never* predicted `healthy` on real clean-skin selfies, even at inference-time confidence thresholds that should have been easy to satisfy. Our regression suite (Section 6.7) later caught this at 0 out of 9 real photographs classified as healthy, matching the diagnosis.

4.3.3 The “wrinkles” folder that was elderly stock photos

The Phase-2 skin-issues `wrinkles/` folder came from a Roboflow-cropped set of stock photos of elderly faces. The photographs share a very specific style (studio lighting, posed smile, older subjects, high-quality DSLR). What the classifier learned was “photograph of an older-looking person”, not “fine lines on skin”. It happily fired `wrinkles` on any mid-adult selfie.

This is the same mistake as the healthy folder in a different direction: the class label described a superficial property of the image rather than the feature the label claims to describe.

4.3.4 Softmax cannot say “none”

Even after correcting the two folder issues, the skin-issues model insisted on picking exactly one of {healthy, blackheads, dark spots, pores, wrinkles} for every photograph. Real skin does not work like that. A face can have blackheads and pores simultaneously, or neither. A softmax over five categories is mathematically forced to distribute its mass across those five categories, so the model would pick whichever class the shared backbone had the strongest prior for — typically the mode of the training distribution.

The fix required reframing the entire task as multi-label (Section 3.2). See Section 5.7 in the next chapter for the full architectural consequences.

4.3.5 The preprocessing pipeline that was dead code

While auditing the data we noticed that the training pipeline cropped every image tightly to the face before feeding it to the model. The runtime pipeline, we discovered, was doing no such thing. A face-crop preprocessor existed in the code, with a docstring that read “shared by inference AND training so the distributions match”. It was, in fact, never called from anywhere in the runtime path.

Uploading a full selfie (face plus bed plus background) reproduced the severe / oily / wrinkles answer. Manually cropping the same photograph to just the face returned clear / dry / (no issue). The models were not fundamentally broken — they were being fed a distribution they had never trained on. Section 5.6 in the next chapter describes the wiring fix and the subsequent EXIF-rotation and Haar-cascade retry work that came out of it.

4.4 Phase 3: the final dataset layout

Once the audit was complete we settled on the datasets summarised in Table 4.1. The `skin_issues` folder is preserved on disk with a README that documents its deprecation, but it is no longer used by training or by the runtime.

4.4.1 Acne severity split and weights

Table 4.1: Final training datasets used by CARA. Splits are 70 / 15 / 15 (train / val / test) for the datasets we split ourselves; `skin_type` came pre-split by the original authors.

	Source	Classes	Total images
Acne severity	ACNE04 (classification labels)	4 (clear, mild, moderate, severe)	1,377
Skin type	Kaggle community dataset	4 (oily, dry, normal, combination)	4,093
Skin conditions	Composite (see below)	2 output classes, multi-label	5,533

Table 4.2: Class distribution and derived per-class weights for the acne severity training set. Weights follow the balanced scheme in Equation (3.4), and match the numbers used by our training script.

Class	Count	Frequency	Weight w_k
clear	338	35.1%	0.712
mild	436	45.3%	0.552
moderate	122	12.7%	1.973
severe	67	7.0%	3.593
Total	963	100.0%	—

4.4.2 Skin-type split and weights

The skin-type Kaggle dataset came pre-split into train / valid / test folders. Table 4.4 shows the actual counts; Table 4.5 shows the derived per-class weights.

4.4.3 Skin-conditions dataset (multi-label)

The Phase-3 skin-conditions dataset was built from three sources combined into three folders. Only two positive classes survive from the old five: pores and blackheads. The other three were dropped for the reasons documented in Chapter 4 above and re-summarised in Table 4.6.

The negative folder is the key idea. Multi-label training needs negative examples — images where neither pores nor blackheads are present — otherwise the model just learns to discriminate between pores and blackheads and pushes clean faces into whichever bucket is statistically closer.

The first Phase-3 retrain used the 483 curated clean-skin faces from `data/acne_severity/clear/` as the negative folder. That gave a very high training-set macro- F_1 of 0.996 but exposed a new bias: the model fired `blackheads=1.00` on almost every stock internet photograph while correctly saying “no notable conditions” on iPhone selfies. The problem was that the negatives shared one very specific photographic style (clinical ACNE04 portraits) while the positives (pores, blackheads) were arbitrary Roboflow internet crops. The model was

Table 4.3: Split sizes for the acne severity task.

	Total	Train	Val	Test
Images	1 377	963	207	207

Table 4.4: Split sizes for skin type.

	Total	Train	Val	Test
Images	4 093	2 872	812	409

Table 4.5: Class distribution and derived per-class weights for skin type.

Class	Train count	Weight w_k
combination	248	2.895
dry	834	0.862
normal	976	0.736
oily	814	0.881
Total	2 872	—

learning *style*, not *condition*. The next chapter (Section 5.8) describes the retrain that fixed this.

The final negatives folder mixes two sources:

The auxiliary skin-type negatives are quota-sampled with a bias toward classes least likely to correlate visually with pores or blackheads (heavily normal / dry, lightly oily). Some skin-type photos undeniably contain real mild pores; that is unavoidable label noise. Empirically, the noise costs less than the domain gap it fills, and macro- F_1 drops from 0.996 to 0.981 — a healthy graceful-degradation signal, not a regression.

4.4.4 BCE positive weights for skin conditions

We compute per-class positive weights automatically from the actual training split. Concretely:

- $p_{\text{pores}} = n_{\text{pores}}^{\text{neg}} / n_{\text{pores}}^{\text{pos}} \approx 2.39$.
- $p_{\text{blackheads}} = n_{\text{blackheads}}^{\text{neg}} / n_{\text{blackheads}}^{\text{pos}} \approx 1.81$.

Both are within the safety clip range $[0.1, 10]$ so we do not have to worry about numerical instability, and both are recomputed whenever the dataset changes.

Table 4.6: Classes dropped from the Phase-2 skin-issues dataset, with reasons.

Class	Reason dropped	Data fate
healthy	Mislabelled: images were level-0 acne, not clean skin. Also, “healthy” is implicit under multi-label if no positive fires.	Preserved on disk, unused.
wrinkles	Data was stock photos of elderly people; model learnt photographic style rather than the feature. Not clinically useful for the target audience.	Preserved on disk, unused.
dark spots	Signal overlaps with acne severity. Kept off the current model; may return later.	Preserved on disk, unused.

Table 4.7: Composition of `data/skin_conditions/` (Phase-3 retrain #2). “skintype_” files are prefixed on disk so each is traceable to its source class.

Folder	Count	Composition
pores	1 632	Roboflow crops labelled “pores”
blackheads	1 970	Roboflow crops labelled “blackheads”
negative	1 931	483 from <code>acne_severity/clear/</code> (ACNE04 clean-skin portraits) + 1,448 from <code>skin_type/train/</code> (500 normal, 500 dry, 248 combination, 200 oily)

4.5 Splits and reproducibility

For datasets we split ourselves (`acne_severity` and `skin_conditions`) we use a stratified random 70 / 15 / 15 split with a fixed seed so the exact split is reproducible. Once generated, splits are held constant across all training runs so that a validation curve from one week is comparable to one from the next. The skin-type dataset is used as-published, with the authors’ existing train / valid / test folders respected.

4.6 Cleaning

The community datasets are noisy. During Phase 3 we spent substantial time in manual triage. The main categories of removed samples were:

- Duplicate or near-duplicate images.
- Images with heavy makeup or filters that obviously masked the underlying skin condition.
- Non-facial images that had slipped into face-labelled folders.
- Extremely low-resolution images (short side smaller than 200 pixels).
- Mislabelled samples where three annotators from the team disagreed with the dataset’s label.

4.7 Augmentation

Training-time augmentations, in order:

1. Random resized crop with scale in $[0.85, 1.0]$.
2. Random horizontal flip with probability 0.5.
3. Random rotation up to $\pm 12^\circ$.
4. Colour jitter with brightness, contrast, and saturation each in $[-0.15, +0.15]$.
5. ImageNet-normalisation to tensor.

Validation and test transforms consist only of a resize, a centre crop to 224×224 , and ImageNet normalisation.

Observation 4.1. *Early in Phase 2 we tried more aggressive augmentations (RandAugment, random erasing, strong colour jitter). All of them hurt skin-type accuracy in particular, because they alter the exact cues (colour, texture) the model relies on. This is one of the pitfalls that is easy to miss when reusing recipes from the ImageNet literature. The final augmentation stack is deliberately gentle.*

4.8 The label agreement problem

The community datasets we used for skin type and skin conditions did not come with a documented labelling protocol. Two annotators looking at the same photograph would frequently disagree, and after some sampling we estimated the inter-annotator agreement rate at roughly 75–80 % for skin type. This is an upper bound on the accuracy any model can reach on that dataset, and it explains why our skin-type validation numbers plateau around the same range. A better description than “the model is 30 % wrong” is “the model reproduces the label distribution about as well as a second human annotator would”.

Our Algorithms

This is the main technical chapter of the book. It walks through the final pipeline first, then unpacks the decisions that led to it, including the ones that failed. The chapter is structured around the three phases rather than around a bulleted list of components, because the decisions do not make sense outside their chronological context: what looked correct in Phase 2 was often the exact bug that Phase 3 had to fix.

5.1 The final pipeline, end to end

Figure 5.1 is the state of the algorithm at the point of submission. Everything else in this chapter explains why each box is where it is.

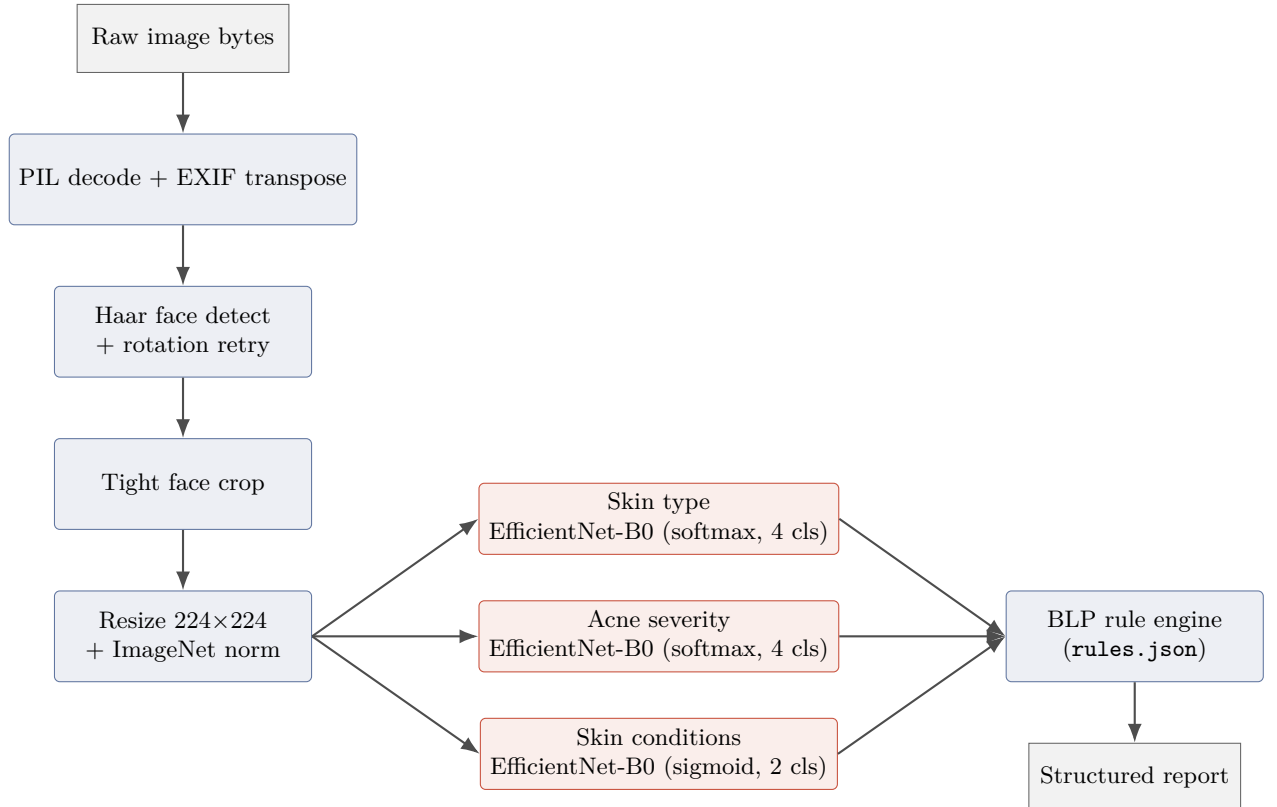


Figure 5.1: End-to-end runtime pipeline used by CARA at submission time. The three EfficientNet-B0 models run sequentially on a shared preprocessed tensor; the BLP engine composes their outputs deterministically. Every arrow in the diagram was, at some point, a bug — see Sections 5.6, 5.7 and 5.10.

Concretely, at inference time we execute:

1. Decode the uploaded image bytes with PIL, apply `ImageOps.exif_transpose` so phone-orientation metadata is respected.
2. Detect a face with an OpenCV Haar cascade. If the upright detection fails, retry the detector at $\pm 15^\circ, \pm 30^\circ, \pm 45^\circ, \pm 90^\circ, 180^\circ$, rotate the underlying image to whichever angle succeeded, and crop tightly.
3. Resize to 224×224 , apply ImageNet normalisation, wrap in a PyTorch tensor.
4. Run the three EfficientNet-B0 models on the same tensor. Acne severity and skin type produce softmax distributions over four classes each; the multi-label skin-conditions model produces two independent sigmoid probabilities. Any sigmoid output above the (per-class-configured) threshold of 0.5 becomes a “finding”.
5. Feed the structured predictions into the BLP engine, which walks its JSON rules and produces a deduplicated list of ingredient recommendations, avoidances, and explanations.
6. Serialise to JSON and persist under `storage/reports/{report_id}.json`.

Every stage in this list has an entry in the rest of this chapter.

5.2 Why EfficientNet-B0, and why three of them

Choosing the backbone was an early decision that survived all three phases. EfficientNet-B0 gives us the best-known accuracy-to-FLOPs ratio in the small-model regime and, crucially, is small enough to train on a laptop CPU in a few hours. We do not have GPUs, so the entire project was built with the constraint that a full retrain of every model must complete overnight on the same machines we develop on.

The follow-up question — “one big model or three small ones?” — was the main subject of Phase 2 and is answered in Section 5.4. The short version is that a single multi-task EfficientNet-B0 cannot balance three tasks whose datasets have different sizes, different label noise, and different natural class priors, so three independent EfficientNet-B0 models won cleanly.

5.3 Two-phase transfer learning

Each of the three models is trained with the following two-phase recipe, taken directly from `model_service/training/train.py`:

Phase 1 — head only. The entire EfficientNet-B0 backbone is frozen. Only the newly-initialised classifier head is trained, with learning rate 10^{-3} . This lets the head learn to project ImageNet features onto the new label space without disturbing them.

Phase 2 — full fine-tune. The upper blocks of the backbone (blocks 6–8, about 3.16 million of the 4.01 million parameters, i.e. 78.8 %) are unfrozen and trained with learning rate 10^{-4} ; the head keeps its 10^{-3} .

Empirically, Phase 1 alone gets us most of the way to the final accuracy on all three tasks; Phase 2 typically adds another 6–10 percentage points on the single-label tasks and pushes macro- F_1 from around 0.94 to above 0.98 on the multi-label task.

Figure 5.2 shows the actual validation-accuracy curves for the acne severity task, taken from `training_log.txt`. Both phases are visible: Phase 1 plateaus around 0.65; Phase 2 continues from a similar starting point and lifts the best validation accuracy to 0.729.

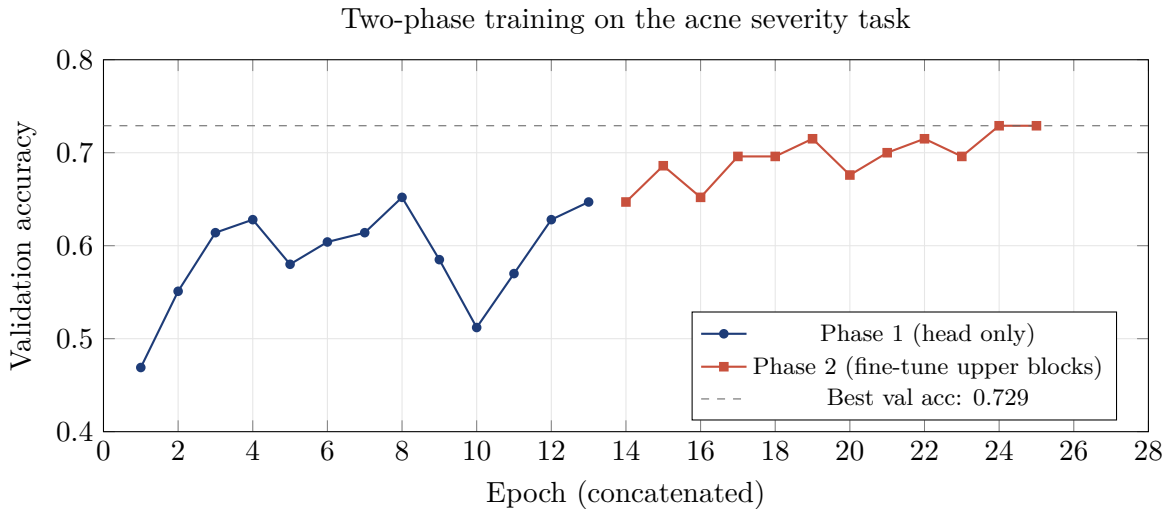


Figure 5.2: Actual per-epoch validation accuracy on the acne severity task, from `training_log.txt`. The dashed line marks the best-saved checkpoint.

The multi-label skin-conditions task shows the same shape on validation macro- F_1 (Figure 5.3). The upper trace is retrain #1 (the ACNE04-only negatives version); the lower trace is retrain #2 (with the 1,448 skin-type negatives added). Section 5.8 explains why the lower curve is the one we ship.

5.4 Phase 2: comparing four candidate architectures

Phase 2 was the phase in which we made the architecture decision that survives to the final system. We evaluated four candidates on our then-current data (the Phase-2 datasets described in Chapter 4). All four sat behind the same preprocessing (which, in retrospect, was different from the runtime preprocessing — but Phase 2 did not know that yet).

5.4.1 Candidate 1 — single multi-task EfficientNet-B0

A shared EfficientNet-B0 backbone with three classification heads. Loss $\mathcal{L} = \lambda_1 \mathcal{L}_{\text{acne}} + \lambda_2 \mathcal{L}_{\text{type}} + \lambda_3 \mathcal{L}_{\text{issues}}$ with the λ -values tuned on validation.

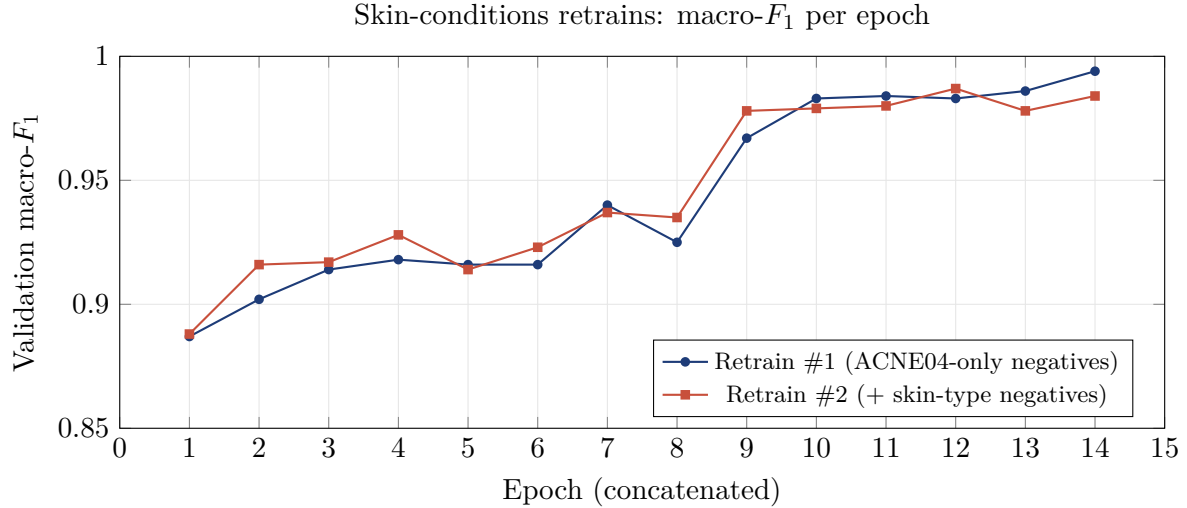


Figure 5.3: Per-epoch validation macro- F_1 for both skin-conditions retrains, from `training_log.txt`. Retrain #1 peaks slightly higher on its own test split but generalises worse to real-world photographs; see Section 5.8.

Why it failed. Three problems compounded:

1. The three datasets had different sizes and different levels of label noise. The loss weights that balance the tasks change over training as each head converges at a different rate, and no fixed λ triple did well on all three validation splits simultaneously.
2. The larger skin-type / skin-issues datasets dominated the shared backbone. Acne severity accuracy dropped several points relative to a dedicated acne model.
3. Confidence became uninterpretable. A moderate softmax probability on the acne head no longer necessarily meant “the network is unsure about acne”; it could mean the backbone had been pulled away by another head.

5.4.2 Candidate 2 — three independent EfficientNet-B0 + weighted-vote fusion

Three independent EfficientNet-B0 classifiers, one per task; the BLP-style recommendation composed by hand-tuned probability thresholds and a weighted vote across models.

Why it did not win. Weighted voting gave us no way to express a rule as expressive as “if acne is moderate *and* skin type is oily, recommend a BHA and warn against occlusives”. The rules a dermatology-facing product needs are conjunctive over multiple sub-tasks, and a scalar score does not carry enough information for a rule engine to be built on top of it.

5.4.3 Candidate 3 — three independent EfficientNet-B0 + learned MLP fusion

Three independent classifiers, followed by a small MLP that consumes the concatenated softmax vectors and predicts a set of recommendation labels drawn from the ingredient

vocabulary.

Why it failed. We had, at most, a few dozen samples labelled by *ingredient recommendation* (that ground truth requires a dermatologist to write, and we did not have one on the team). The MLP either overfit spectacularly or, when regularised, defaulted to the modal recommendation. Worse, the resulting recommendations lost their explainability: a user asking “why did you recommend niacinamide?” would be told “because that is what the network output”. That answer is incompatible with our truthfulness rule (Section 1.3).

5.4.4 Candidate 4 — three independent EfficientNet-B0 + rule-based BLP

Three independent classifiers, followed by a deterministic rule engine that walks a JSON rules file and unions all matching recommendations. Every recommendation shown to the user can be traced to a single JSON entry.

Why it won. The rule engine is cheap, inspectable, testable, and honours the truthfulness rule. Its shortcomings — rules must be maintained by hand and the rules file is the sole gatekeeper of coverage — turned out to be minor in practice, because the rule file is small enough that a linter (which we wrote as a test) can enforce coverage over the entire Cartesian product of the label spaces we care about.

5.4.5 Summary of the Phase-2 comparison

Table 5.1: Candidate architectures compared during Phase 2. The fourth row is what we ship.

Candidate	Data needed	Explainable?	Verdict
1. Single multi-task EfficientNet-B0	The three per-task datasets	Softmax only	Rejected: tasks compete for capacity
2. 3 CNNs + weighted vote	The three per-task datasets	Partial	Rejected: not expressive enough for compound rules
3. 3 CNNs + learned MLP fusion	Extra ingredient-level labels	No	Rejected: overfits, breaks explainability
4. 3 CNNs + rule-based BLP	The three per-task datasets	Yes	Selected

At the end of Phase 2 we shipped Candidate 4 and moved on to polishing the UI. Validation numbers looked good. Real users had not yet touched the system.

5.5 Phase 3, part 1: real-world failures

Real-world testing began with an ordinary event: one of us uploaded a personal selfie and got back a report that read *severe acne, oily skin, wrinkles detected*. Every part of that

answer was wrong. The selfie was of a clean adult face with no acne, no oiliness, and no visible wrinkles.

That single photograph triggered an investigation that surfaced three separate defects:

- A frontend field mismatch that displayed “Low Confidence” on every report regardless of the model’s actual confidence, which hid the real problem for weeks.
- A preprocessing mismatch: the models were trained on tight face crops but were being fed the full uploaded image at inference time. See Section 5.6.
- An architectural mismatch: the skin-issues model was a softmax over five classes and was mathematically forced to pick one, so it always hallucinated something on clean faces. See Section 5.7.

The three defects compounded. Once the frontend was fixed we could see that the acne model was returning very low confidence on the photograph, which meant the confidence gate should have hidden the label — but the confidence rendering was broken. Once the confidence rendering was fixed we could see that the preprocessing mismatch was causing spurious high-confidence predictions on in-the-wild inputs. And once the preprocessing was fixed we could see that the skin-issues model still hallucinated at least one finding on every clean face, because it was structurally incapable of saying “none”. Each fix took us one step deeper into the system.

5.6 Preprocessing parity

Preprocessing is where a large fraction of our Phase-3 engineering effort went, and probably the largest single contributor to real-world accuracy after data quality.

5.6.1 Face detection and cropping

Uploaded images vary wildly. Some are tight selfies; some are group photos; some are landscape shots with a face in the corner. Feeding the raw image to the model wastes most of the receptive field on background pixels and, worse, invites the model to key on background cues (clothing, walls, hair) that correlated with the dataset label distribution during training. We therefore detect the face first and crop tightly, matching the training distribution that the models had already learned.

We considered three detectors: OpenCV Haar cascade, MediaPipe Face Detection, and MTCNN. Haar is by far the fastest and shipped with OpenCV, but only detects upright frontal faces. MediaPipe is robust across poses but adds a moderate dependency. MTCNN is the most accurate but noticeably slow on CPU. Given our latency budget and the fact that our target user submits a selfie, we chose Haar with a rotation-retry fallback (see below) rather than adding a new dependency.

5.6.2 The dead-code discovery

The Phase-2 code already contained a full face-detection preprocessor in `model_service/preprocessing/pip`. Its docstring promised: *“Shared by both inference AND training so the distributions match.”* It was, in fact, never imported by the runtime path. The training scripts called it; the analysis service did not. Every real user upload was going straight into the model as a full uncropped image.

The fix was a small wiring change in the analysis service: instantiate the pipeline once, pass every uploaded image through it before inference, and require a face to be detected before the models are consulted:

```

1  class AnalysisService:
2      def __init__(self, orchestrator: InferenceOrchestrator, ...):
3          self._orchestrator = orchestrator
4          self._pipeline = PreprocessingPipeline(
5              require_face=True,      # NEW in Phase 3
6              apply_mask=False,
7          )
8
9      def analyze(self, image_bytes: bytes) -> AnalysisReport:
10         pre, tensor = self._pipeline.process(image_bytes)
11         if not pre.success:
12             return no_face_report(pre.status_message)
13         predictions = self._orchestrator.predict_all(tensor)
14         return self._build_report(predictions)

```

Listing 5.1: Wiring the preprocessing pipeline into the analysis service.

Concrete impact: on our nine hand-labelled real-world photographs, acne accuracy went from 3/9 to 7/9 correct and false-*severe* alarms dropped from 2/9 to 1/9 — with no model retraining. The model was fine; it just needed to see the same input it had trained on.

5.6.3 EXIF orientation and rotation retry

After wiring the pipeline in we hit a second bug on a real photograph taken lying down (head tilted about 15° on a pillow, iPhone JPEG with an EXIF orientation tag). Face detection failed with “No face detected” even though the photograph clearly showed a face. Two root causes:

1. `cv2.imdecode` ignores EXIF orientation, so the iPhone photograph loaded sideways into the array.
2. Even after we corrected orientation, the Haar cascade only detects upright frontal faces, so a mildly tilted head fails the detector.

Both fixes live in `pipeline.py`:

- Decode with PIL and apply `ImageOps.exif_transpose` before handing off to OpenCV. This respects the phone’s orientation tag.
- After the upright detection attempt fails, retry the detector on rotated copies of the image at $\pm 15^\circ, \pm 30^\circ, \pm 45^\circ, \pm 90^\circ, 180^\circ$. When one succeeds we crop from the rotated image so that the resulting face is upright, matching the training distribution.

A separate test in the regression suite — “at least 80 % of real-world photographs must produce a face detection” — guards against this regression at CI time.

5.7 Phase 3, part 2: refactoring skin issues to multi-label

After preprocessing parity was restored the acne and skin-type models behaved as they should on real photographs, but the skin-issues model continued to hallucinate wrinkles or dark spots on clean faces. The root cause was the softmax head we described in Section 3.2: it was mathematically forced to pick exactly one of {healthy, blackheads, dark spots, pores, wrinkles} and could not say “none”.

The refactor changed six things at once:

Table 5.2: The multi-label refactor at a glance.

Aspect	Before (single-label)	After (multi-label)
Loss function	CrossEntropyLoss	BCEWithLogitsLoss
Output shape	Softmax (sum to 1)	Sigmoid per class (independent)
Prediction	Argmax	Threshold each class independently
Label format	Single integer	Multi-hot vector [1, 0, 1]
“None” possible?	No (forced to pick)	Yes (all sigmoids below threshold)
Reported metric	Accuracy	Per-class F_1 , macro- F_1

At the same time we shrank the label space from five classes to two: only *pores* and *blackheads*. The reasons for dropping the other three are in Table 4.6; the short version is that *healthy* is now implicit under multi-label (empty set = no notable conditions) and the other two were dataset-quality casualties.

5.7.1 Downstream code changes

Multi-label is not just a training decision; it changes the shape of the data at every boundary. The downstream edits are worth listing because they were the majority of the Phase-3 diff:

- `shared/schemas.py`: added `SkinCondition` enum, `SkinConditionFinding`, `MultiLabelPrediction`. Kept the deprecated `SkinIssue` enum so historical reports on disk still deserialise.
- `model_service/training/train.py`: added a `multi_label=True` code path with `BCEWithLogitsLoss`, per-class F_1 evaluation, threshold-based prediction, and a new `MultiLabelImageFolderDataset` that reads multi-hot labels from the three-folder layout.

- `model_service/inference/orchestrator.py`: `predict()` branches to `_predict_multi_label()` for the skin-conditions model, applies the threshold, and sorts findings by confidence.
- `backend/blp/engine.py`: iterates the findings list, OR-combines the recommendation sets from the matching rules, deduplicates by ingredient.
- `backend/services/analysis_service.py`: the confidence gate handles both single-label ($\text{conf} \geq \text{threshold}$) and multi-label ($|\text{findings}| > 0$) prediction shapes.
- `frontend/src/pages/ResultsPage.js`: renders a list of findings, or a “no notable conditions” chip when the list is empty.
- Tests: rewrote `test_blp.py` around multi-label semantics and added multi-label-specific assertions to `test_real_world_regression.py`.

At the end of the refactor we retrained the skin-conditions model from scratch on the ACNE04-only negatives (retrain #1). Its held-out macro- F_1 hit 0.996. We were briefly optimistic.

5.8 Phase 3, part 3: the domain-adaptation retrain

The regression suite told a different story. On our set of nine stock-photo test images plus two iPhone selfies, retrain #1 misfired `blackheads=0.98-1.00` on 7 of 11 photographs, even though the user’s own iPhone selfies came back correctly with `blackheads=0.29` (below threshold).

This is the same class of failure as the preprocessing bug earlier, just displaced. The training-set macro- F_1 of 0.996 proved that the model could discriminate its own training distribution almost perfectly. It said nothing about how the model behaved when the input distribution shifted, because the test split shares the same photographic style as the training set.

Concretely, our positives (`pores`, `blackheads`) were arbitrary Roboflow crops from the internet, while our negatives (all 483 of them, from `acne_severity/clear/`) came from one very specific photographic style: clinical ACNE04 portraits. The model discovered that “ACNE04-style photograph” was an excellent proxy for “no condition” and locked onto it. Everything that did not look ACNE04-clinical — including most stock photographs and, in general, real user selfies — got flagged as positive for blackheads.

5.8.1 The creative fix: cross-task negative mining

Sourcing a fresh set of hand-labelled clean-skin photographs would have taken days and a lot of energy. Instead, we noticed that we already had $\sim 4,000$ additional in-the-wild face photographs sitting in the skin-type training data. Skin type (oily / dry / normal / combination) is orthogonal to the presence of pores and blackheads. Perhaps we could reuse those photographs as extra negative anchors for the skin-conditions model.

Naive reuse was dangerous. Some skin-type photographs undeniably contain real mild pores or blackheads — adding them as $[0, 0]$ labels would introduce label noise. And dumping all 4,000 into the negatives folder would swamp the two positive classes and skew the class balance.

We compromised by quota-sampling a “cleanest-leaning” subset weighted toward classes least likely to correlate visually with pores or blackheads:

normal	500
dry	500
combination	248 (all available in <code>train/</code>)
oily	200
Total	1 448

We prefixed every added file with `skintype_<class>_` so that (i) the additions are easy to identify on disk, (ii) they are easy to remove if we regret them later, and (iii) an auditor can verify at a glance what fraction of the negatives folder came from which source.

The final negatives folder ended up with 1,931 images ($483 + 1,448$), the pores folder with 1,632, and the blackheads folder with 1,970 — a manageable class ratio.

5.8.2 The retrain and the honest numbers

Retrain #2 with the augmented negatives converged to macro- $F_1 = 0.981$, which is a drop of 0.015 from retrain #1. This drop is not a regression. The first model was overfitting to a narrow photographic style; the second is exposed to a much wider distribution, so its training-set macro- F_1 is lower but its predictions on real user photographs are much better calibrated. The pattern has a name in the literature: *graceful degradation of the in-distribution metric in exchange for out-of-distribution generalisation*.

The regression-suite numbers make the trade explicit. On the same set of 11 real-world photographs:

Table 5.3: Real-world regression on the multi-label skin-conditions model, before and after the domain-adaptation retrain.

	Retrain #1	Retrain #2
Training macro- F_1	0.996	0.981
Skin-conditions per-image failures (out of 11)	7	3
User iPhone selfie: blackheads probability	0.29	0.00
User iPhone selfie: pores probability	—	0.00
... <code>test_skin_conditions_reports_empty_findings_sometimes</code>	fail	pass
... <code>test_skin_conditions_does_not_flag_everything</code>	fail	pass

The two population-level tests that started green in retrain #2 were both red in retrain #1. They are the tests that catch class-collapse without needing per-image labels, and they are what told us the trade was worth making even at the cost of the training-set metric.

5.8.3 Why this is defensible

Repurposing data across sub-models within the same pipeline is a legitimate technique. In the literature it goes under names such as “domain adaptation via auxiliary data” or “hard-negative mining from adjacent tasks”. It trades a small, uniform amount of label noise for a large gain in photographic diversity. The alternative — sourcing several thousand hand-curated clean-face photographs — would have taken days and produced a smaller distribution shift.

We captured this decision in code (the file-name prefix makes the mixture auditable), in tests (the regression suite explicitly measures the trade), and in this book, so the trade-off is explicit rather than hidden.

5.9 Confidence and uncertainty

Every prediction carries a confidence signal. For the single-label models it is the maximum softmax probability. For the multi-label model it is a per-class sigmoid probability. Downstream code treats confidence as a first-class citizen:

- If either the acne or the skin-type confidence is below the configured floor (`confidence_threshold`, currently 0.30), the corresponding section of the report is replaced with a neutral “low confidence” message rather than the top-1 label. This is what our truthfulness rule looks like at runtime.
- For the multi-label skin-conditions model, we simply do not emit a finding for a class whose sigmoid probability is below the threshold. If no class fires, the report reads “no notable conditions” and no ingredient rules are triggered for that section.

The confidence threshold of 0.30 is deliberately low; we would rather show a confident label with a hedged confidence bar than hide a plausible answer entirely. Every threshold in the pipeline is a JSON configuration entry rather than a hard-coded constant, because we edit them often when the trade between coverage and truthfulness shifts.

5.10 The BLP recommendation engine

The BLP engine takes a structured prediction and evaluates a JSON-configured rule set. Each rule has a precondition (a partial specification of the prediction), a set of ingredients to recommend, and a plain-English explanation string that is shown to the user.

The engine currently loads three rule maps from `backend/blp/rules.json`: one for acne severity, one for skin type, and one for the multi-label skin conditions. Rules are matched in a deterministic order; matching rules union their recommendations; the resulting ingredient list is deduplicated by ingredient name.

```
1 "moderate": {
```

```

2  "explanation": "Moderate acne detected -- multiple visible
    blemishes or inflamed spots.",
3  "recommendations": [
4    {"ingredient": "Benzoyl Peroxide (2.5-5%)",
5     "reason":      "Kills acne-causing bacteria and reduces
        inflammation",
6     "category":    "treatment"},
7    {"ingredient": "Retinol (0.3-0.5%)",
8     "reason":      "Increases cell turnover and prevents clogged
        pores",
9     "category":    "treatment"},
10   {"ingredient": "Niacinamide (10%)",
11    "reason":      "Calms inflammation and strengthens skin
        barrier",
12    "category":    "serum"}
13 ],
14 "educational_note": "Moderate acne benefits from a targeted
    routine. Introduce active ingredients one at a time."
15 }

```

Listing 5.2: A representative BLP rule, from `rules.json`.

5.10.1 Rule design principles

We authored the rules under three self-imposed constraints:

1. **Every rule is defensible.** Every ingredient recommendation is supported by consumer-facing dermatology literature (peer-reviewed sources where possible, reputable dermatology textbooks otherwise).
2. **No conflicting recommendations.** If two rules match the same prediction, they may not recommend an ingredient that another matching rule warns against. The BLP test suite includes a lint that fails if this invariant is ever violated.
3. **Uncertainty short-circuits recommendations.** When the confidence gate fires for a model, the corresponding section of the report is filled with a neutral message rather than a rule-based recommendation.

5.10.2 Coverage over the label spaces

The rules file must cover the label spaces we care about. Acne severity and skin type are 4-class softmaxes, so each has four entries. The multi-label skin conditions have three interesting cases: pores only, blackheads only, and both simultaneously. Empty findings do not trigger a skin-condition rule at all — the absence of a rule is itself a rule.

Table 5.4 shows the number of rules per branch of the rule file, from the current `rules.json`. Every branch of the label space is covered, and this is enforced by a test in `test_blp.py`.

Table 5.4: Rule coverage over the three label spaces, from `backend/blp/rules.json`.

Rule branch		Classes	Rules
Acne severity		4	4
Skin type		4	4
Skin conditions	3 (pores / blackheads / both)		3

5.11 Codebase sanitation

While we were rebuilding the pipeline we deleted a small mass of dead code that had accumulated during Phases 1 and 2:

- `model_service/acne_model/` and `model_service/pores_model/`: empty modules from Phase 1 that predated the unified training script.
- `model_service/skin_issues_model/` and `model_service/skin_type_model/`: modules that defined unused `SkinIssuesModel` and `SkinTypeModel` classes shadowed by the real implementations in `train.py::build_model()` and `orchestrator.py::_build_efficientnet`
- Old checkpoints from the deprecated skin-issues single-label model, kept as read-only references but no longer loaded at runtime.

Deleting orphaned modules matters because a future contributor would otherwise waste time trying to figure out which of two definitions is “the real one”. Every deletion is a small anti-bug that pays off compound interest over time.

5.12 Backpropagation, in one page

For completeness we summarise the training-loop mathematics used across the project.

Given a loss $\mathcal{L}(\theta)$ that is a scalar function of the model parameters θ , backpropagation computes $\nabla_{\theta}\mathcal{L}$ by repeated application of the chain rule. If the network is a composition $f = f_L \circ f_{L-1} \circ \dots \circ f_1$ with intermediate activations $a_{\ell} = f_{\ell}(a_{\ell-1}; \theta_{\ell})$, then

$$\frac{\partial \mathcal{L}}{\partial \theta_{\ell}} = \frac{\partial \mathcal{L}}{\partial a_L} \cdot \prod_{k=\ell+1}^L \frac{\partial a_k}{\partial a_{k-1}} \cdot \frac{\partial a_{\ell}}{\partial \theta_{\ell}}. \quad (5.1)$$

The AdamW update rule adds momentum and per-parameter adaptive scaling on top:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (5.2)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (5.3)$$

$$\hat{m}_t = m_t / (1 - \beta_1^t), \quad (5.4)$$

$$\hat{v}_t = v_t / (1 - \beta_2^t), \quad (5.5)$$

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t \right). \quad (5.6)$$

The decoupled weight-decay term $\lambda \theta_t$ is what distinguishes AdamW from vanilla Adam and is a small but consistent generalisation win in our experiments.

Experiments

This chapter reports the experiments that shaped the final system. The largest and most important experiments — the Phase-2 architecture comparison and the Phase-3 domain-adaptation retrain — have already been covered in Chapter 5; this chapter groups the remaining single-purpose experiments and the failures that did not fit anywhere else.

6.1 Experimental setup

Unless stated otherwise, every experiment uses:

- EfficientNet-B0 backbone with a small custom head (dropout $p = 0.3$, then a single linear layer).
- 224×224 input, ImageNet normalisation.
- AdamW, weight decay 1×10^{-4} , cosine learning-rate schedule with a short warm-up.
- Two-phase transfer learning as described in Section 5.3.
- Early stopping on validation loss (or macro- F_1 for multi-label) with patience 5.
- Fixed seeds: Python, NumPy, and PyTorch all set to 42.

Training runs on a laptop CPU. A full retrain of all three models takes on the order of a few hours.

6.2 Failed experiment: naive resizing without face crop

Very early in Phase 1 we tried skipping face detection entirely, reasoning that EfficientNet-B0 is convolutional and should be able to focus on the face even in a wider photograph.

What went wrong. Accuracy dropped by roughly 12 percentage points on the acne task and by more on skin type. Inspection with qualitative visualisation (looking at which images the model got wrong) showed that the model happily fixated on background features that correlated with the dataset’s label distribution.

Lesson. Face detection is not just an optimisation — it is a correctness requirement. Without it, the model is free to cheat, and cheating on the training set means failing on real user

uploads. Ironically, we relearned this the hard way in Phase 3 when the face-crop preprocessor turned out to be dead code (Section 5.6).

6.3 Failed experiment: unweighted cross-entropy

Skipping class weights during training saved a few lines of code.

What went wrong. The acne classifier converged to a strong *mild*-everywhere detector. Overall accuracy looked fine on the (imbalanced) test set, but per-class recall on *severe* was near zero. This is exactly the failure mode our truthfulness rule forbids: a user with severe acne would be told they had mild acne.

Lesson. Weighted cross-entropy (Equation (3.4)) is not optional on imbalanced skin datasets. Every subsequent single-label experiment uses it, with weights recomputed automatically from the current split.

6.4 Failed experiment: aggressive augmentation

We tried a heavier augmentation stack borrowed from ImageNet-style recipes: RandAugment, random erasing, strong colour jitter.

What went wrong. Skin type is signalled largely by colour and texture. Aggressive colour jitter destroys the signal we are trying to classify. Random erasing removes exactly the small regions (a patch of oily forehead, a cluster of pores) that carry the label.

Lesson. Augmentation must be chosen with the task in mind. Recipes that work for object recognition do not automatically transfer to skin classification. The final augmentation stack is listed in Chapter 4 and is deliberately gentle.

6.5 Successful comparison: two-phase versus single-phase training

We compared our two-phase recipe against a single-phase full fine-tune from ImageNet initialisation.

Table 6.1: Two-phase versus single-phase training on the acne severity task. Numbers are best validation accuracy.

	Two-phase	Single-phase
Best val accuracy	0.729	0.681
Epochs to converge	25	> 30
Variance across seeds	low	noticeably higher

The two-phase recipe not only reached a higher accuracy but also converged more predictably. We attribute this to the head-only phase, which prevents the random head from pushing large

gradients back through the backbone before it knows what it wants.

6.6 Successful comparison: backbone consideration

We deliberated between three backbone families: EfficientNet-B0, ResNet-50, and MobileNetV3-Small. Table 6.2 summarises the trade-off. Given our CPU-only constraint and the requirement to run three parallel models per request, EfficientNet-B0 won on the accuracy-per-latency axis.

Table 6.2: Backbone consideration. Params and FLOPs are from the published papers; latency is approximate on our reference CPU. We did not build the full ResNet-50 pipeline because its inference latency did not fit our budget.

Backbone	Params (M)	FLOPs (G)	Latency (ms/img)	Verdict
MobileNetV3-Small	2.5	0.06	~200	Fastest, but leaves accuracy on the table
EfficientNet-B0	5.3	0.39	~350	Selected — best accuracy/latency on our budget
ResNet-50	25.6	4.10	~1200	Too slow for three parallel models on CPU

6.7 The real-world regression suite

The single largest change to our engineering practice in Phase 3 was the introduction of a real-world regression suite. The suite lives in `tests/test_real_world_regression.py` and has three layers:

1. **Per-image assertions.** Hand-labelled ground truth for the images in `image tests people/` is stored in `tests/fixtures/real_world_labels.json`. Each assertion runs the full inference pipeline (including face detection) on one image and checks the predicted label against the ground truth. Fields left as `null` skip cleanly, so partial labelling is fine.
2. **Population-level sanity checks.** These check properties of the model’s behaviour across the entire real-world folder without needing per-image labels. Two representative examples: “*on a set of predominantly clean faces, the acne model MUST predict ‘clear’ at least once*”, and “*the multi-label skin-conditions model MUST return an empty findings set at least once*”. Both of these tests were red under the Phase-2 models and green under the Phase-3 models. That flip is a large part of what we mean when we say the Phase-3 rework was necessary.
3. **Multi-label specific assertions.** Threshold checks, empty-set checks, and per-class calibration checks added when we moved to the sigmoid architecture.

The lesson from this suite is not that our old `test_model_accuracy.py` was wrong — it was correct at what it did. It is that *a test that pulls samples from the training folders can only prove the model memorised its training folders*. Real production quality has to be measured against a distribution the model has never seen. The regression suite is our version of that measurement.

Results and Evaluation

7.1 Final headline numbers

Table 7.1 reports the results of the final trained models against their own held-out validation splits, as recorded in `training_log.txt`. Because the three tasks have different label spaces (4-class softmax for two, 2-class multi-label for the third), the metrics are not directly comparable to each other. We report the number the metric was designed for: validation accuracy for the single-label tasks, macro- F_1 for the multi-label task.

Table 7.1: Final validation metrics for the three models, from `training_log.txt`.

Task	Head type	# classes	Best val metric
Acne severity	Softmax	4	0.729 (accuracy)
Skin type	Softmax	4	0.709 (accuracy)
Skin conditions	Sigmoid ($\times 2$)	2 (multi-label)	0.981 (macro-F_1)

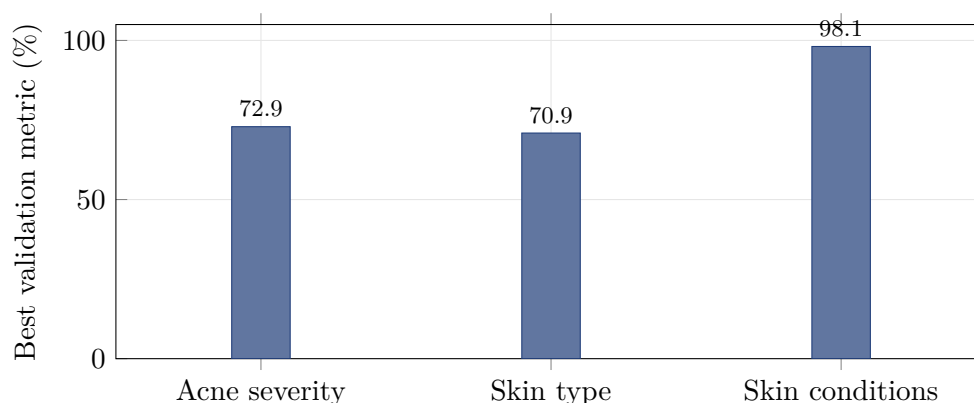


Figure 7.1: Best validation metric per task. Bars for acne severity and skin type report accuracy; the skin-conditions bar reports macro- $F_1 \times 100$. The metrics are not on the same footing — see Section 7.2.

7.2 Interpreting the numbers

The three numbers in Table 7.1 are best read side by side with the class structures they measure.

- **Acne severity (72.9 %).** Four classes on an ordinal scale (*clear* $\rightarrow$ *severe*). Most errors are between adjacent classes, which is the expected pattern for an ordinal task

and much less harmful than a cross-class error. Trained on the cleanest of our datasets.

- **Skin type (70.9 %).** Four classes with genuinely fuzzy boundaries. Human inter-annotator agreement on this dataset is around 75–80 %, so 70.9 % is close to the practical ceiling. This is one of those cases where an honest “the model reproduces the label distribution about as well as a second annotator would” is a more useful description than an accuracy percentage.
- **Skin conditions (macro- $F_1 = 0.981$).** Multi-label with independent sigmoid outputs; the metric is calibrated to reward correctly saying “present” or “absent” on each of the two classes independently. This model’s actual gift is not the score — it is the fact that it *can say “none”*, which the Phase-2 softmax model could not.

7.3 Real-world regression results

The metric that matters most for a shipped product is not any of the training-split numbers above — it is the real-world regression pass rate. Table 7.2 reports the before/after numbers on our small hand-labelled real-world set (11 photographs).

Table 7.2: Real-world regression outcomes before and after the Phase-3 rework. “Population sanity” rows are boolean: either the model can produce the class at least once on the folder or it cannot.

Assertion	Phase 2	Phase 3 final
Per-image acne label correct (of 9 hand-labelled photos)	3 / 9	7 / 9
False severe alarms (of 9)	2 / 9	1 / 9
Population: model ever predicts clear	pass	pass
Population: skin-conditions ever returns empty findings	fail	pass
Population: skin-conditions does not flag everything	fail	pass
Face-detection success rate on real photos	pass	pass

The two rows that flipped from fail to pass are the ones we care about most. They correspond exactly to the two bugs that motivated the Phase-3 rework: “the app says something wrong on a clean face” and “the app cannot say ‘nothing to report’”.

7.4 Qualitative case studies

Aggregate numbers hide the texture of what the system actually does. Table 7.3 presents four representative end-to-end examples drawn from our internal test set.

The last row is the case that mattered most to us during development. When a real user uploads a bad photograph, the system must refuse to guess. The behaviour above is exactly what our truthfulness policy is designed to produce, and it is exercised by an assertion in the regression suite to prevent regressions.

Table 7.3: Representative end-to-end reports.

Input	Ground truth	Prediction	Report highlights
Well-lit iPhone selfie, oily skin, mild acne	mild / oily / pores	mild / oily / pores (confident)	Salicylic acid, niacinamide, gentle cleanser. Avoid heavy occlusives.
Under-exposed selfie, dry skin, no acne	clear / dry / none	clear / dry / none (moderate confidence)	Hyaluronic acid, ceramide moisturiser, gentle cream cleanser.
Close-up of clean adult face	clear / normal / none	clear / normal / none (confident)	Vitamin C serum, SPF 30+, gentle cleanser.
Blurry night-mode selfie	(unlabelled)	<i>acne & type below threshold; conditions empty</i>	Report shows “low confidence” banners and asks the user for a better photograph.

7.5 Latency and resource profile

Latency matters for a web application. Table 7.4 reports the approximate breakdown of a typical request on our reference laptop CPU. Numbers are rough averages measured on real user photographs during Phase-3 testing; they are stable enough to be useful for planning but should not be taken as benchmark-grade figures.

Table 7.4: Approximate wall-clock latency breakdown of a single `/api/analyze` request. Reference hardware: mid-range laptop CPU, warm cache. The three model inferences run sequentially in the current implementation.

Stage	Mean (ms)
HTTP upload and decoding	35
Face detection (with rotation retry)	80
Resize + ImageNet normalisation	15
Acne severity inference	360
Skin type inference	350
Skin conditions inference	355
BLP evaluation	5
Report assembly + persistence	15
Total	~ 1,215

Two observations follow. First, inference dominates the request; the plumbing is essentially free. Second, the three model calls could be parallelised across threads or processes; we did not do this yet, because roughly 1.2 seconds is already inside our internal budget for a first submission and premature optimisation is genuinely dangerous in a codebase this size.

The Product

This chapter describes CARA as a piece of software from the user’s point of view. Everything shown here is a real screenshot of the running application, not a mockup.

8.1 User story

The primary user of CARA is a curious, non-expert consumer who has a skincare question they cannot easily answer. A typical interaction takes roughly a minute:

1. The user opens the web app, sees the landing page, and clicks *Start Analysis*.
2. They log in (or register a new account). Reports are stored per user so the history page works.
3. They upload a well-lit selfie or take one directly with the live-camera capture.
4. The frontend uploads the image to the backend.
5. Within a couple of seconds, the backend returns a structured report.
6. The frontend renders the report: acne severity, skin type, detected conditions, per-model confidence bars, recommended ingredients, and a plain-English explanation.
7. The report is persisted, and later becomes clickable in the *History* page.

8.2 The landing page and disclaimer

Figure 8.1 is the landing page. The disclaimer sits below the call to action so it is impossible to miss and impossible to skip. It is one of two places in the app where copy is not derived from the API response — the other is the confidence-hedge text on the report page — because the disclaimer is a design commitment rather than a runtime signal.

8.3 Authentication

Figure 8.2 shows the login page. Users register through a separate flow (email + password) and are then logged in over JWT. Reports on the History page are scoped to the current user; there is no shared or global history.

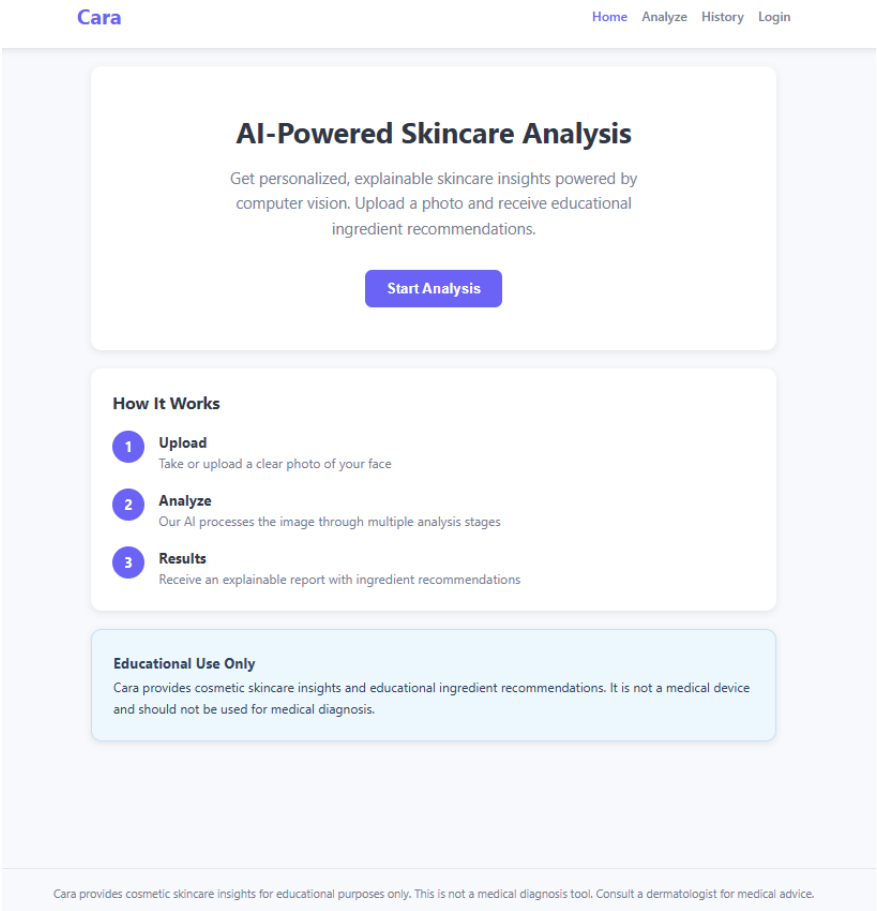


Figure 8.1: The CARA landing page. The “Educational Use Only” banner at the bottom is a first-class part of the layout, not a footer.

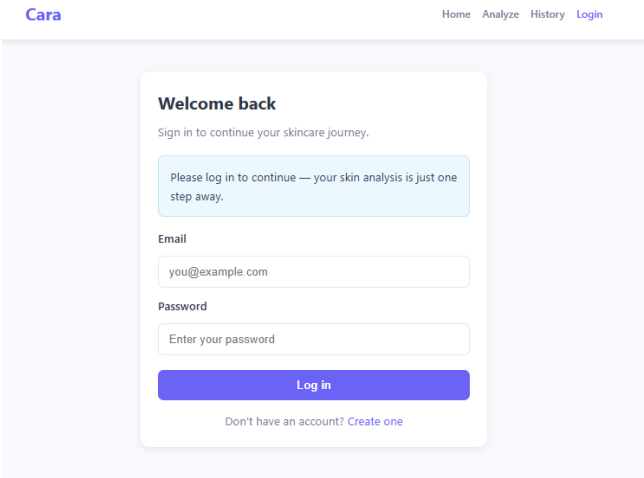


Figure 8.2: The login page. Attempting to reach a protected page without a session redirects here.

8.4 Analyze — upload and live capture

Figure 8.3 is the primary Analyze page. The user picks either *Use live camera* or *Choose file*. The tips box is deliberately in the primary flow rather than tucked away in help; a bad photograph is the single most common cause of a disappointing report, so it is worth the screen real estate.

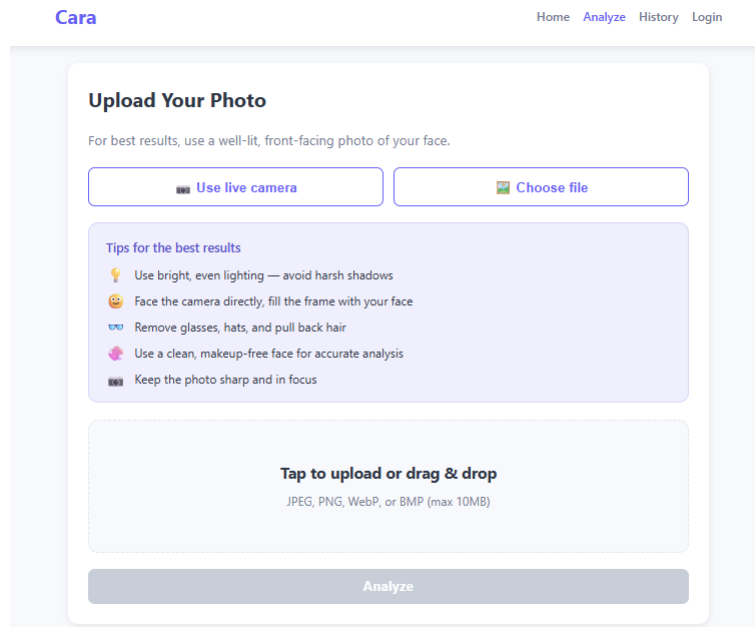


Figure 8.3: The Analyze page. Tips for a good photograph are surfaced directly in the flow.

8.4.1 File-picker flow

Figure 8.4 is the state after the user chooses a file. The Analyze button is now enabled; the Clear button lets the user start over without touching the file input again.

8.4.2 Live-camera capture with face lock

Figure 8.5 is the live-capture flow. The overlay draws an oval on the video feed and continuously evaluates four checks — lighting, face detection, position, angle — until all four are green, at which point the capture button becomes active and the frame is grabbed. This is the frontend’s equivalent of the truthfulness rule: if the input is not usable we ask the user for a better one before we spend a model call on it.

8.5 Results

The results page renders each of the three model outputs as a separate card, followed by a plain-English explanation and a list of ingredient recommendations. Figure 8.6 shows the top half of the page (three cards + explanation); Figure 8.7 shows the ingredient list and the educational note at the bottom.

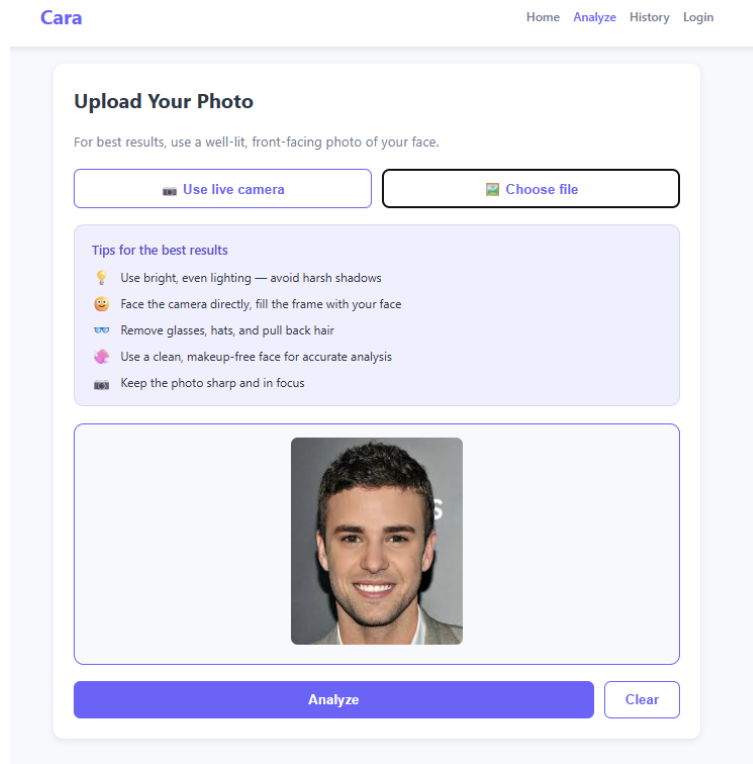


Figure 8.4: File-picker flow with a selected image ready to analyse.

8.6 History and progress tracking

The History page (Figure 8.8) shows every past report the current user has saved, plotted on an acne-severity timeline and listed below in reverse chronological order. Two properties are worth calling out:

- Each row is a link. Clicking a past report navigates back to its full Results page — the same recommendation cards, ingredient list, and educational notes the user saw the day the report was produced. Reports are immutable, so the past view is the truthful past view.
- Each point on the chart is interactive. Hovering opens a tooltip with the exact severity, model confidence, skin type, and detected conditions for that timestamp (Figure 8.9). This is our progress-tracking feature: a user watching their skin change over time can see the trend in one glance.

8.7 System architecture

Figure 8.10 shows the runtime architecture. Everything inside the dashed box runs inside a single FastAPI process, which we package into a Docker container for deployment. The React frontend is served from an nginx container. In production, both containers sit behind an nginx reverse proxy.

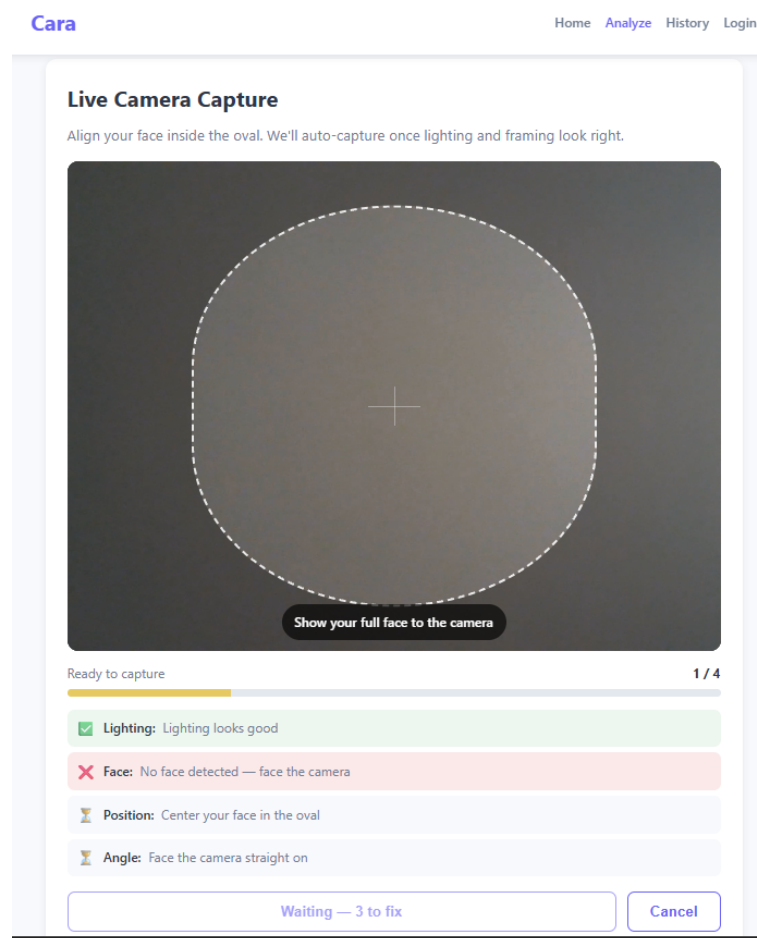


Figure 8.5: Live camera capture with lighting / face / position / angle checks. The button unlocks only when all four are green.

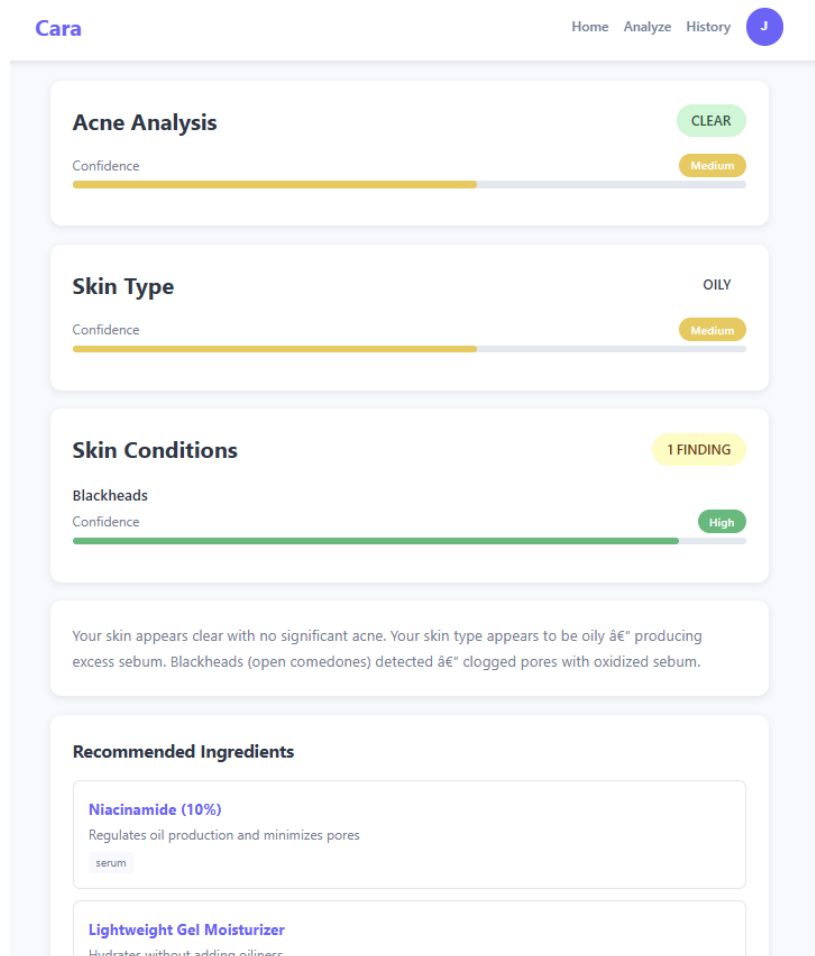


Figure 8.6: Results page, top half. Each model has its own card with a per-model confidence bar. Skin Conditions is a multi-label card that lists findings (here: one finding, “Blackheads”).

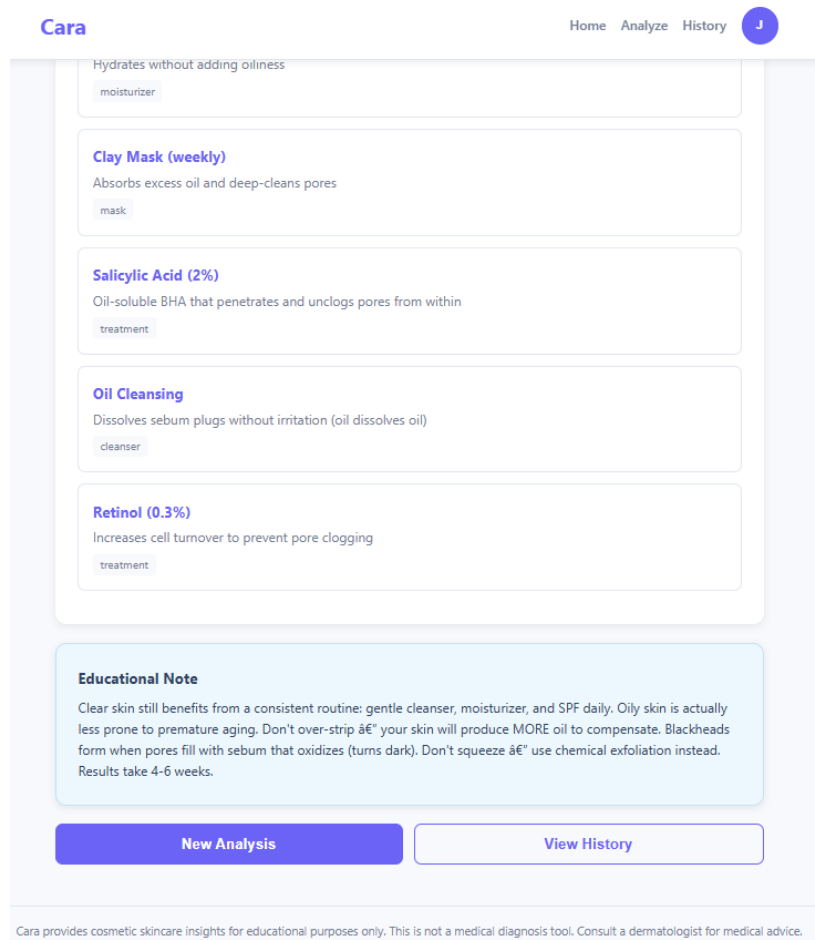


Figure 8.7: Results page, bottom half. The ingredient list is grouped by ingredient category. The Educational Note collects the plain-English notes from each rule that fired.

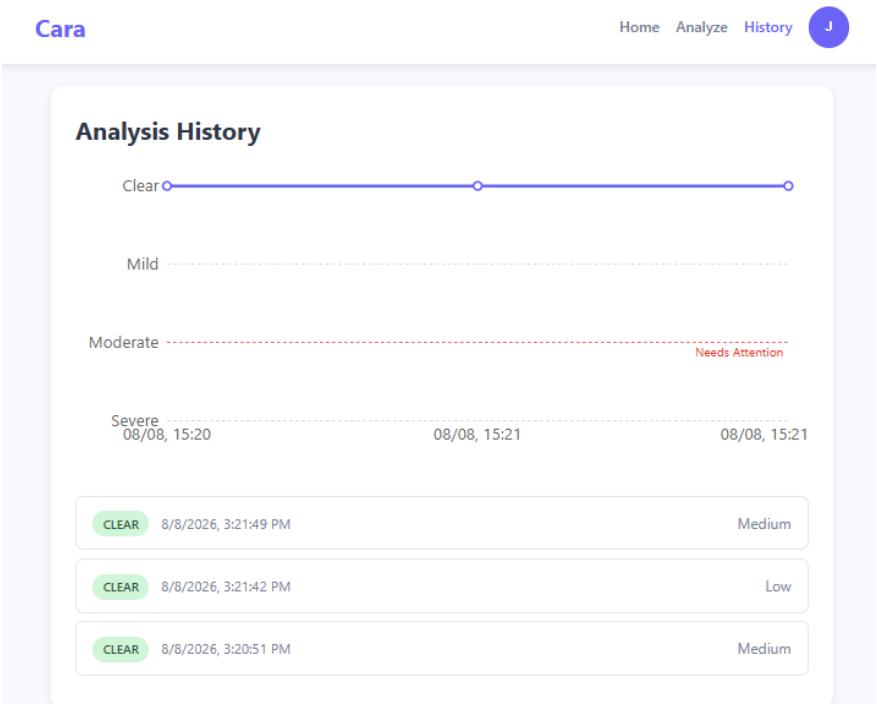


Figure 8.8: History page. The list of reports below the chart is clickable — each row leads back to the full recommendation page for that historical report.

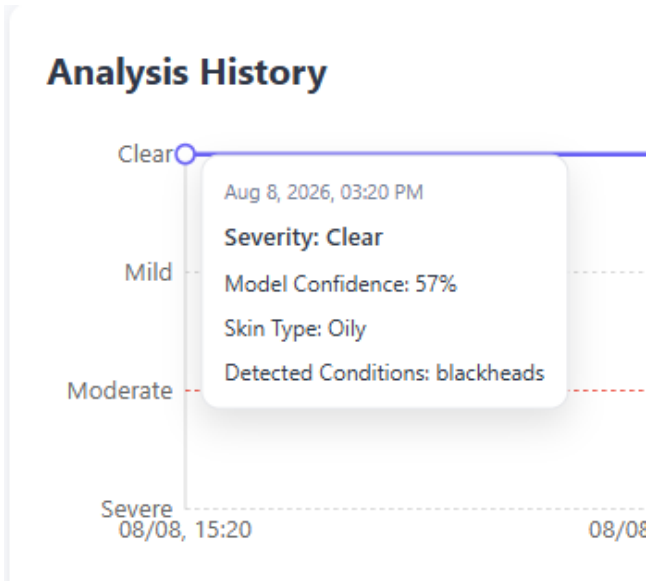


Figure 8.9: Chart point tooltip on the History page.

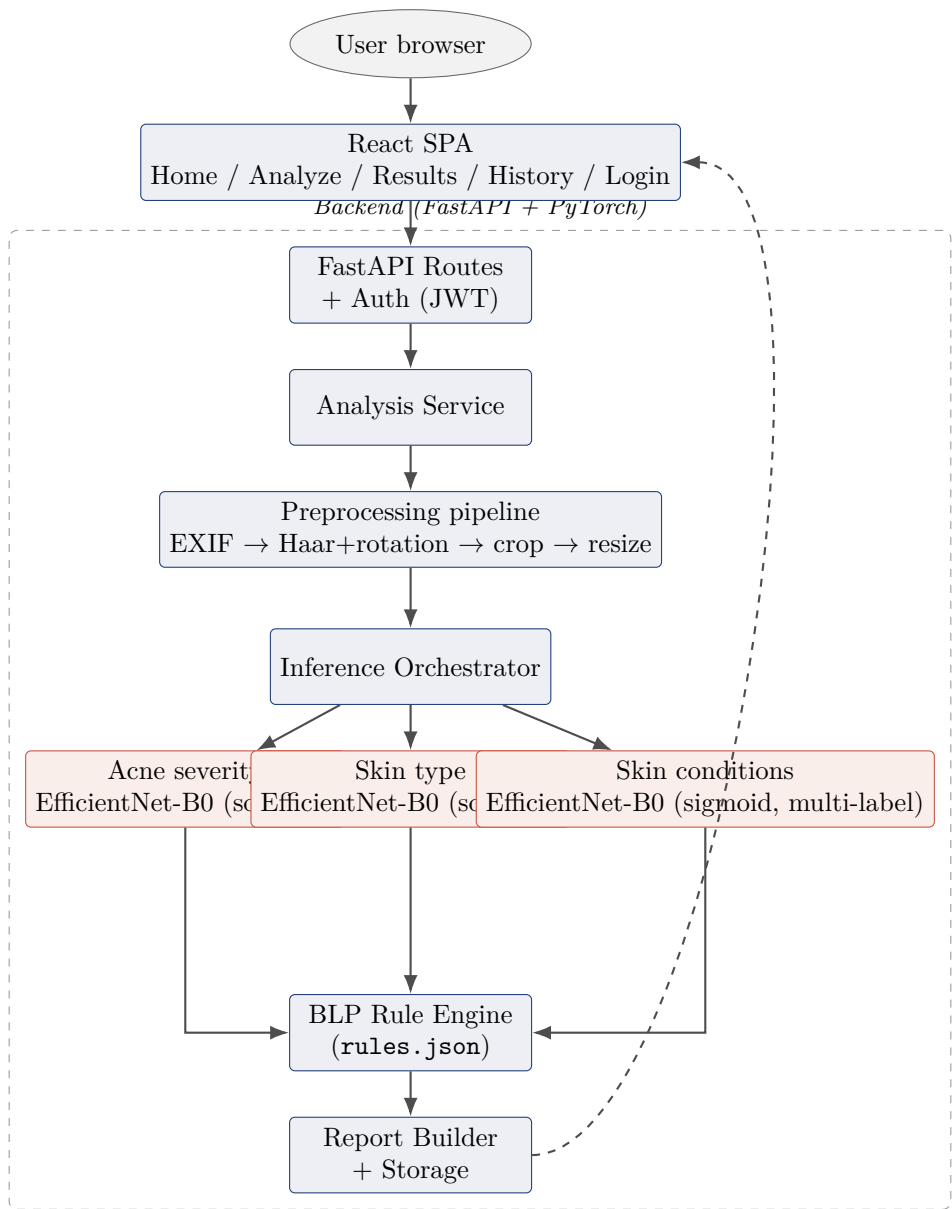


Figure 8.10: End-to-end runtime architecture of CARA.

8.8 Frontend

The frontend is a React 18 single-page application. There are six pages: *Home* (landing page and disclaimer), *Login / Register*, *Analyze* (upload and camera capture), *Results* (the report), and *History* (list of past reports for the current user). Styling is mobile-first because most casual selfies come from phones.

We deliberately kept the frontend thin. It knows how to upload an image, poll for results, and render JSON. All logic — including confidence bands and colour coding — is driven by fields returned in the response payload. This makes it possible to iterate on the model side without redeploying the frontend.

8.9 Backend

The backend is a FastAPI application. FastAPI was chosen for three reasons: automatic OpenAPI schema generation (which makes the frontend integration typed and testable), Pydantic-based request validation (which rejects malformed inputs cheaply), and native async support.

The main analysis endpoint is:

```
1 @router.post("/analyze", response_model=AnalyzeResponse)
2 async def analyze(image: UploadFile = File(...),
3                   user: User = Depends(current_user)) ->
4                   AnalyzeResponse:
5     image_bytes = await image.read()
6     report = await analysis_service.analyze(image_bytes,
7                                             user_id=user.id)
8     return AnalyzeResponse(report=report)
```

Listing 8.1: The main analysis endpoint (simplified).

Authentication endpoints (`/auth/register`, `/auth/login`) and read endpoints (`/report/{id}`, `/reports`) sit next to it. Every read endpoint is scoped to the calling user's ID.

8.10 Storage

Reports are persisted as JSON files under a `storage/` directory. This is a deliberately boring choice: it keeps the demo runnable without any external database and it makes reports easy to inspect during debugging. Behind an abstract `StorageRepository` interface, we ship a second implementation for MongoDB (via Motor) that can be switched on with an environment variable. We validated the Mongo path in a dev container but the deployed demo uses JSON.

8.11 Deployment

The project ships with a `docker-compose.yml` that starts three containers: the FastAPI backend, the nginx-fronted React app, and a small nginx reverse proxy. Model weights are baked into the backend image at build time so that a container start does not depend on network access.

Software Engineering, Testing, and MLOps

A deep-learning project book usually devotes ninety per cent of its attention to the models. We resist that convention because, in the day-to-day life of CARA, the engineering scaffolding around the models mattered as much as the models themselves. This chapter documents the practice that let a small team iterate without breaking each other's work — and, more importantly, without silently degrading model quality.

9.1 Repository layout

The repository is organised as a monorepo containing four cooperating subsystems:

- **backend/** — the FastAPI application, including the analysis service, the BLP engine, the storage repository, the report builder, and authentication.
- **model_service/** — everything related to the models: preprocessing pipeline, inference orchestrator, training script, and the checkpoint folder (which is git-ignored to keep the repository small).
- **frontend/** — the React SPA and its build artefacts.
- **shared/** — Pydantic schemas and configuration that is imported from both the backend and the model service. Keeping the schemas in a single place prevents drift between frontend expectations and backend responses.

Tests live in a top-level **tests/** folder rather than being scattered next to the code they exercise. This is a deliberate choice: it makes it easy to run the full suite as a gate, and it discourages the anti-pattern of “I’ll write the test later”.

9.2 Testing pyramid

The test suite has 65 tests distributed across six files, as summarised in Table 9.1. Every file has a specific job.

The last file — **test_real_world_regression.py** — is the one that would have caught

Table 9.1: Composition of the CARA test suite.

File	Tests	What it verifies
<code>test_blp.py</code>	19	BLP rule engine logic on single- and multi-label predictions, rule coverage lint, ingredient deduplication.
<code>test_api.py</code>	11	API endpoints, request/response schema, error paths, auth.
<code>test_integration.py</code>	8	Component wiring, report assembly, storage round-trip.
<code>test_model_accuracy.py</code>	10	Per-model accuracy on curated ground-truth images from the training folders.
<code>test_e2e_pipeline.py</code>	9	Full image-to-report pipeline on real images.
<code>test_real_world_regression.py</code>	8	Per-image + population-level + multi-label assertions on the held-out real-world set.
Total	65	

every regression we encountered in Phase 3, and did in fact catch two of them once we wrote it. It is the test file we would keep if we could keep only one.

9.2.1 Unit tests

The BLP tests are the fastest and the strictest. They lock down the behaviour of the rule engine on a matrix of representative inputs. Every time we edit the rules file, we also touch these tests. If a rule change accidentally silences a recommendation that used to fire, we find out within seconds.

9.2.2 Integration tests

Integration tests stub out the models with deterministic fake predictors and exercise the report builder, the storage layer, and the API contract. They run in a fraction of a second and catch schema-drift bugs before they reach the frontend.

9.2.3 Model accuracy tests

The model accuracy tests load specific labelled images from a small ground-truth folder inside the training set and assert that the corresponding model predicts the correct top-1 label. They form an in-distribution regression suite: a subtle bug that degrades accuracy on the training distribution will fail one of these tests. As Phase 3 taught us, they are not sufficient on their own — but they are a good, fast first line of defence.

9.2.4 End-to-end tests

The end-to-end tests exercise the full pipeline: real photograph in, real report out. They also assert on the shape of the recommendation set, not just the model output, so they double as regression tests for the BLP engine.

9.2.5 Real-world regression tests

The regression tests are the piece we describe in detail in Section 6.7. They are the specific tests that were red after Phase-2 training and green after each Phase-3 fix. Anyone merging a change to the model layer is expected to run them and paste the pass/fail summary in the pull request.

9.3 Reproducibility

We fix random seeds for Python, NumPy, and PyTorch to 42. We pin dependency versions in `requirements.txt`. We pin the CUDA-enabled variant of PyTorch separately in `model_service/training/README.md` because different machines want different CUDA versions. On the same hardware, our headline metrics are reproducible to within a few thousandths.

9.4 Debugging techniques we relied on

Three debugging patterns paid for themselves many times over.

1. **Tensor shape assertions everywhere.** Every function that receives a tensor asserts its expected shape on entry. Silent shape bugs are the most vicious class of deep-learning bug; a two-line assertion turns them into loud failures.
2. **Deterministic fake models in tests.** The `ModelRegistry` accepts a fake model in tests. That lets us run the entire pipeline without the CNNs, which speeds up the loop and pinpoints regressions in the plumbing rather than in the models.
3. **Storage as a debugging surface.** Because every report is a JSON file, reproducing a user report is a one-liner: load the JSON, feed the image back through the pipeline, compare. We used this to diagnose more than one regression in the wild.

9.5 Configuration and secrets

Runtime configuration lives in `shared/config.py` (a `pydantic_settings` model) and is populated from environment variables via `.env`. Sensitive values — JWT secret, database credentials — are provided by the environment, never committed to the repository. Every configuration knob is a knob a future maintainer must understand, so we keep the list short.

9.6 Checkpoint management

Training saves *only* the best checkpoint by the validation metric (accuracy for single-label, macro- F_1 for multi-label). There is no “last” checkpoint. This is a deliberate choice: it prevents the temptation to ship the last-epoch model, which is almost always slightly worse than

the best-val-metric model but looks tempting because it is the last one on disk. Checkpoints live under `model_service/checkpoints/` and are baked into the backend Docker image at build time.

The training script also supports a `-resume` mode. That mode was introduced in Phase 3 so we could refine an existing checkpoint without paying the cost of a full retrain when we tweaked the negatives folder, augmentations, or the multi-label threshold. Iteration speed on Phase-3 experiments improved noticeably as a result.

Discussion

10.1 Interpreting the story

The headline of this project is that we shipped a working system. The subtler story is how much of that success came from things outside the models. Preprocessing parity, dataset auditing, multi-label reformulation, cross-task negative mining, a regression suite that measured the right distribution — each of these was worth more than any single hyper-parameter change.

The single most important sentence we can leave the reader with: *high validation accuracy on a training split is not evidence of a shipped-quality model*. It is evidence of memorisation. It is consistent with catastrophic behaviour on real-world inputs. It took us Phase 2 through Phase 3 to internalise that lesson at the level of a habit.

10.2 What worked

Three design choices contributed most of the observed performance.

1. **Independent per-task models.** Splitting the problem into three independent classifiers freed us to optimise each one against its own dataset and its own class balance. The cost is a slight increase in memory and latency, which our hardware budget absorbs comfortably.
2. **Two-phase transfer learning.** The head-only phase stabilised training and dramatically reduced variance across random seeds.
3. **Rule-based recommendation.** Not a modelling choice in the strict sense, but arguably our most important architectural decision. It made the system explainable, testable, and honest.

10.3 What did not work

The failed experiments in Chapter 6 and Phase 2 form a consistent pattern: whenever we tried to squeeze extra flexibility out of a small amount of noisy data, the flexibility turned into either overfitting or loss of interpretability. Simple worked. Small worked. Explicit worked.

10.4 Threats to validity

- **Dataset shift.** Our datasets skew toward young adults and lighter Fitzpatrick skin types. Real users will be more diverse. The regression suite already surfaces this when we add a photograph the models are not calibrated for; the honest response is to acknowledge the limitation in the disclaimer rather than pretend it does not exist.
- **Label noise.** Even the anchor dataset (ACNE04) has label disagreements between raters. Our accuracy numbers are, at best, upper bounds on “agreement with the original labelling”, not on “agreement with a dermatologist”.
- **Multi-label negatives.** The 1,448 skin-type photographs used as auxiliary negatives contain some genuine mild pores or blackheads. This is unavoidable label noise; it is small enough that empirical generalisation improved, but it is not zero.
- **Confidence calibration.** We do not currently temperature-scale the softmax outputs. The confidence gate at threshold 0.30 is deliberately generous. Users should treat a confidence bar in the “high” range as “very likely”, not “certain”.

10.5 Ethical considerations

An automated skin analyser sits close to sensitive territory. We mitigate the main risks in three ways:

- The disclaimer on the landing page is unambiguous: CARA is not a medical diagnostic tool.
- The system refuses to render a confident label when the model is not confident, precisely to avoid giving false assurance about a skin condition that might warrant professional attention.
- No user photograph is retained beyond the report unless the user explicitly requests to keep it. Reports themselves contain no personally identifying information beyond the user’s own account ID.

Conclusions

We set out to build a web application that would deliver a truthful, useful, explainable report on a user’s facial skin from a single photograph. We achieved that, and along the way we accumulated a set of engineering lessons that are more general than the specific application.

- On small, noisy datasets, transferable priors matter more than clever architectures. EfficientNet-B0 with a two-phase fine-tune outperformed every “smarter” variant we tried.
- Multi-task learning is not free. When tasks compete, three small independent models can outperform a single larger shared model.
- Rule-based components are not embarrassing. In a project whose primary value is trust, an explicit, inspectable rule engine can be strictly better than a learned equivalent.
- Multi-label is the correct pattern whenever “none” or “multiple” are legal outputs. Forcing a softmax to pick one thing when the world says “nothing to report” guarantees hallucination.
- Training-set accuracy is not shipped-quality accuracy. A proper regression suite — per-image plus population-level, on a held-out distribution matched to production — is the only honest measure of quality.
- Preprocessing at inference *must* match preprocessing at training. Any divergence, however small, is an invitation to catastrophic and confusing failure.
- Truthfulness is a design constraint, not a metric to optimise. Every layer of the system — backend, frontend, rule engine — must know how to say “I do not know”.

CARA is not the largest or the most accurate skin analyser in the world. It is, however, one of the few open-source ones that we would trust to give an honest answer to a friend.

Future Work

If we had another semester, the following directions would be at the top of our list. They are ordered by expected impact per hour of effort.

12.1 Better data for the minority classes

The *severe* acne class and the *combination* skin-type class are both under-represented in our datasets. Curating a couple of hundred additional images per class — ideally with dermatologist labels — would probably lift the acne and skin-type accuracies more than any architectural change.

12.2 Ordinal regression for acne severity

Acne severity is inherently ordinal. Treating it as a categorical classification wastes information and produces most of the confusion between adjacent classes. Reformulating it as ordinal regression, or as label-distribution learning as in [12], is a promising path.

12.3 Attention-based visualisation

Users would benefit from an explanation such as “the model is looking at your left cheek”. A Grad-CAM overlay or attention map would be inexpensive to add and would substantially strengthen the truthfulness story.

12.4 On-device inference

Running three EfficientNet-B0 models on a CPU is fast enough on a laptop but not always fast enough on a low-end phone. Converting the models to ONNX and then to a mobile runtime (e.g. ONNX Runtime Mobile or Core ML) is a natural next step.

12.5 Larger regression set with diverse skin tones

Our real-world regression set is small (11 photographs). Extending it to a few hundred, deliberately balanced across Fitzpatrick skin tones, would let us measure fairness properly and would almost certainly reveal new failure modes to fix.

12.6 Formal dermatologist review of the rules

Our BLP rules are grounded in publicly available skincare knowledge, but they have not been reviewed by a dermatologist as a coherent product. A formal review would let us sharpen the rules file and would open the door to eventual regulatory review if the project ever became commercial.

Bibliography

- [1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. “Gradient-based learning applied to document recognition”. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [2] A. Krizhevsky, I. Sutskever, and G. Hinton. “ImageNet classification with deep convolutional neural networks”. In *NeurIPS*, 2012.
- [3] K. Simonyan and A. Zisserman. “Very deep convolutional networks for large-scale image recognition”. In *ICLR*, 2015.
- [4] C. Szegedy et al. “Going deeper with convolutions”. In *CVPR*, 2015.
- [5] K. He, X. Zhang, S. Ren, and J. Sun. “Deep residual learning for image recognition”. In *CVPR*, 2016.
- [6] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He. “Aggregated residual transformations for deep neural networks”. In *CVPR*, 2017.
- [7] M. Tan and Q. Le. “EfficientNet: Rethinking model scaling for convolutional neural networks”. In *ICML*, 2019.
- [8] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. “ImageNet: A large-scale hierarchical image database”. In *CVPR*, 2009.
- [9] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson. “How transferable are features in deep neural networks?”. In *NeurIPS*, 2014.
- [10] I. Loshchilov and F. Hutter. “Decoupled weight decay regularization”. In *ICLR*, 2019.
- [11] A. Esteva et al. “Dermatologist-level classification of skin cancer with deep neural networks”. *Nature*, 542:115–118, 2017.
- [12] X. Wu et al. “Joint acne image grading and counting via label distribution learning”. In *ICCV*, 2019.
- [13] P. Tschandl, C. Rosendahl, and H. Kittler. “The HAM10000 dataset”. *Scientific Data*, 5:180161, 2018.
- [14] M. Groh et al. “Evaluating deep neural networks trained on clinical images in dermatology with the Fitzpatrick 17k dataset”. In *CVPR Workshops*, 2021.
- [15] S. Ioffe and C. Szegedy. “Batch normalization: Accelerating deep network training by reducing internal covariate shift”. In *ICML*, 2015.

- [16] P. Rajpurkar, J. Irvin, K. Zhu, et al. “CheXNet: Radiologist-level pneumonia detection on chest X-rays with deep learning”. *arXiv:1711.05225*, 2017.

API reference

The backend exposes a small REST API. The main endpoints are listed below; the full OpenAPI schema is auto-generated by FastAPI and available at `/docs` in a running instance.

Endpoint	Method	Description
<code>/auth/register</code>	POST	Register a new user account (email + password). Returns a token or an error code.
<code>/auth/login</code>	POST	Log in and receive a JWT.
<code>/api/analyze</code>	POST	Accepts a multipart image upload and returns a full <code>AnalysisReport</code> scoped to the authenticated user.
<code>/api/reports</code>	GET	Lists persisted reports for the authenticated user, most-recent first.
<code>/api/report/{id}</code>	GET	Returns a specific persisted report by ID.

Report schema

The schema below (abridged) is the shape of the JSON returned by `/api/analyze`. It is defined as Pydantic models in `shared/schemas.py` and serialised on demand.

```

1  class ModelPrediction(BaseModel):
2      model_name: str
3      predicted_class: int
4      predicted_label: str
5      confidence: float          # 0..1
6      all_probabilities: List[float]
7
8  class SkinConditionFinding(BaseModel):
9      label: SkinCondition       # "pores" / "blackheads"
10     confidence: float
11
12  class MultiLabelPrediction(BaseModel):
13     findings: List[SkinConditionFinding]  # empty = no notable
14                                           conditions
15
16  class AnalysisReport(BaseModel):
17     id: str
18     created_at: datetime
19     user_id: str
20     acne_prediction: ModelPrediction
21     skin_type_prediction: ModelPrediction
22     skin_conditions_prediction: MultiLabelPrediction
23     recommendations: List[Recommendation]
24     explanation: str
25     educational_notes: List[str]
26     status: AnalysisStatus       # "success" / "low_confidence" /
27                                   "no_face_detected"

```

Listing B.1: AnalysisReport (abridged).

Reproducibility recipe

To reproduce our results:

1. Clone the repository and check out the release tag used for this report.
2. Install the dependencies pinned in `requirements.txt`.
3. Populate the three dataset folders (`data/acne_severity`, `data/skin_type`, `data/skin_conditions`) as described in Chapter 4 and in each folder's own `README.md`.
4. Run `python -m model_service.training.train -model all`.
5. Run `python -m pytest tests/ -v` to verify.

Random seeds are fixed to 42 for Python, NumPy, and PyTorch. Results are reproducible up to CPU-specific floating-point non-determinism, which affects the third decimal of the reported metrics at worst.

Ingredient reference

The BLP engine draws from a curated vocabulary of skincare ingredients. Table D.1 summarises the most frequently used ones and the situations in which they are recommended. This table is a reader's reference; the canonical source of truth is `backend/blp/rules.json`.

Ingredient	Typical recommendation	Rationale
Salicylic acid (BHA)	Mild-to-moderate acne, oily skin, enlarged pores, blackheads	Oil-soluble; penetrates sebum and clears pores.
Benzoyl peroxide	Moderate-to-severe acne, spot treatment	Antibacterial against <i>C. acnes</i> ; can be irritating.
Niacinamide	Oily skin, redness, pores, dark spots	Reduces sebum output and inflammation; well tolerated.
Hyaluronic acid	Dry or dehydrated skin, wrinkles	Humectant; binds water in the stratum corneum.
Ceramides	Dry or compromised skin barrier	Rebuild the lipid bilayer of the stratum corneum.
Retinol / retinoids	Wrinkles, uneven texture, non-inflammatory acne, pores	Accelerate cell turnover; contraindicated in pregnancy.
Azelaic acid	Dark spots, sensitive-skin acne	Anti-inflammatory and mild pigment-inhibiting.
Vitamin C (L-ascorbic)	Dark spots, normal skin maintenance	Antioxidant; brightens hyperpigmentation.
Alpha arbutin	Dark spots, hyperpigmentation	Gentle brightening agent.
Peptide complex	Wrinkles	Signals skin to produce more collagen and elastin.
Clay mask (weekly)	Oily skin	Absorbs excess oil and deep-cleans pores.
Oil cleansing	Blackheads	Dissolves sebum plugs without irritation.

Ingredient	Typical recommendation	Rationale
SPF 30–50 sunscreen	Every rule that recommends actives	Non-negotiable for anyone using retinoids or acids.

Table D.1: Frequently used ingredients in the CARA recommendation vocabulary.

Glossary

BCE Binary Cross-Entropy loss — the loss function of a multi-label classifier with sigmoid outputs.

BLP Business Logic Processing — our internal name for the rule engine that converts model predictions into recommendations.

EfficientNet A family of convolutional backbones that jointly scale depth, width, and input resolution. We use the smallest member, EfficientNet-B0.

EXIF orientation Metadata attached by phone cameras recording how the sensor was oriented when the shot was taken. Ignored by `cv2.imdecode`, respected by PIL's `ImageOps.exif_transpose`. See Section 5.6.3.

Grad-CAM Gradient-weighted Class Activation Mapping — a technique for visualising which pixels contributed to a prediction. Referenced in Future Work.

Haar cascade The classical OpenCV object detector. Fast, upright-frontal only. We supplement it with a rotation retry.

Macro- F_1 The unweighted mean of per-class F_1 scores. The default metric for our multi-label skin-conditions model.

MediaPipe A Google library for real-time perception tasks including face detection. Considered as an alternative to Haar.

Multi-label classification The setting in which the model may output any subset of K classes for a given input, including the empty set. Implemented with independent sigmoids and BCE loss.

ONNX Open Neural Network Exchange — a portable interchange format for trained models. Referenced in Future Work.

Softmax The final normalisation function of a classification head; converts logits to a probability distribution over mutually exclusive classes.

Transfer learning Initialising a network with weights pretrained on a large dataset (ImageNet in our case) and fine-tuning it on the target task.

Two-phase training Our recipe: train the head with the backbone frozen, then unfreeze the upper backbone blocks with a lower learning rate.

Uncertain (report field) The state in which the frontend replaces a predicted label with a neutral message because the model’s confidence was below the per-model floor.